

SONIC PINNACLE: ADVANCING AGE-LINKED SPEECH ANALYSIS THROUGH AI



Supervisor:

Engr. Danish Arif Siddiqi

Submitted From:

Usama Arif (RP20-EE-406)

Noor Mohsin (RP20-EE-430)

Muhammad Faiz Raza (RP20-EE-434)

Ghufran Ahmed Barcha (RP20-EE-435)

Institute of Electrical, Electronics and Computer Engineering

Univeristy of the Punjab ,Lahore

December 2024

ACKNOWLEDGEMENTS

All credit belongs to Almighty Allah who, by delivering us just an amount of His infinite wisdom, enabled us to successfully complete this difficult assignment. Our project supervisor, Danish Arif, deserve our sincere gratitude. We owe them a great deal for their guidance and support during this process.

We are also grateful to our parents and extended family, who have always encouraged us and instilled in us the virtues of integrity and diligence. Lastly, I offer my regards and blessings to all of those who supported me in any respect during the completion of this project

Abstract

A child's development of speech is an important feature of his or her cognitive and social learning abilities. The identification of speech disorders at an early stage is critical to early intervention, but the current practice of relying on conventional speech and language assessment techniques that require speech-language pathologists to evaluate the child's speech and language development is laborious, arbitrary, and variable. This work aims to solve the problem of having a way of examining children's speech and language development that is automatic, accurate, and preferably efficient through the use effective deep learning techniques. Among the recommendation made in the current work, there is application of Convolution Recurrent Neural Network (CRNN) model which compiles Convolution Neural Networks (CNNs) for feature extraction and Recurrent Neural Networks (RNN) for sequence modeling, and particularly incorporating CRNN model takes the Mel-frequency cepstral coefficients (MFCCs), a standard feature in most of the speech recognition systems, as the input and categorize the children's speech samples from ages of 3 to 12 years into developmental stages. The data set in this study is divided into the train, validation and test with 60%, 20% and 20% respectively for the realistic evaluation of models. or an automated, accurate, and efficient system to assess speech development in children, utilizing advanced deep learning (DL) techniques. We propose a new approach using a Convolutional Recurrent Neural Network (CRNN) architecture, combining the strengths of Convolutional Neural Networks (CNNs) for feature extraction and Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) layers, for sequence modeling. The CRNN model processes Mel-frequency cepstral coefficients (MFCCs), a commonly used feature in speech recognition tasks, to classify speech samples of children aged 3 to 12 years into distinct developmental stages. The dataset used for this study was divided into training (60%), validation (20%), and test (20%) sets to ensure a robust evaluation of the model's performance. To modify the quality of the input data, we used normalization and MFCC extraction methods for data preprocessing. We used different hyperparameters for training and fine-tuning of the model, such as batch size, learning rate, epoch count etc., while applying early stopping for regularization to avoid overfitting. Our experimental evaluation encourages that the proposed CRNN model provides a substantial enhancement in dependent, precise, reject and F1-score across all ages of the participants. ROC curve analysis and confusion matrices complement the results and again

prove how the model is accurate in classification of speech patterns, especially between similar ages. It was seen that while GRU layers enabled a higher speed of the training process, LSTM layers helped to bring a slight increase in the accuracy of the identification of long-term dependencies in the sequences of speech. The system is designed to provide a real time response making it useful for speech-language pathologists, educators, and parents who in turn may use it in searching for signs of speech development .This research work benefits the populace through making use of artificial intelligence in the assessment of the speech development of children without the need for extensive training.

Table of Contents

CHAPTER I	11
INTRODUCTION.....	11
1.1. BACKGROUND	11
1.2. PROBLEM STATEMENT	12
1.3. PROPOSED SOLUTION.....	13
1.4. SIGNIFICANCE	14
1.5. RESEARCH OBJECTIVE	15
1.6. RESEARCH QUESTIONS	15
1.7. ALIGNMENT WITH SUSTAINABLE DEVELOPMENT GOALS.....	16
CHAPTER II.....	18
LITERATURE REVIEW	18
2.1. INTRODUCTION.....	18
2.2. OVERVIEW	20
2.3. NEURAL NETWORK	20
2.3.1. How do Neural Network Works.....	21
2.4. TYPES OF NEURAL NETWORKS.....	22
2.5. RECURRENT NEURAL NETWORK (RNN).....	24
2.5.1. Working of RNN	25
2.6. LONG SHORT-TERM MEMORY (LSTM).....	28
2.6.1. Logic Behind LSTM.....	29
2.6.2. Architecture of LSTM	30

2.7. GATED RECURRENT UNIT (GRU).....	33
2.7.1. <i>How GRU's work</i>	34
2.7.2. <i>Architecture</i>	35
2.8. CONVOLUTIONAL RECURRENT NEURAL NETWORKS (CRNN).....	37
2.8.1. <i>Architecture of CRNN</i>	37
CHAPTER III	40
METHODOLOGY	40
3.1.OVERVIEW	40
3.2. BLOCK DIAGRAM	40
3.3. DATA COLLECTION	41
3.3.1. <i>Speech Dataset Description</i>	41
3.4. MODEL ARCHITECTURE	45
3.4.1. <i>CRNN Overview (CNN + GRU/LSTM)</i>	47
CHAPTER IV.....	50
SYSTEM DESIGN.....	50
4.1 OVERVIEW	50
4.2 FRONT-END:REACT	51
4.2.1 <i>Features of the React Front End</i>	52
4.3 BACK-END: FLASK.....	57
4.3.1 <i>Flask API Routes</i>	57
4.3.2 <i>Deep Neural Network Model</i>	59
4.4 FIREBASE INTEGRATION.....	60
4.4.1 <i>Firestore Authentication</i>	60
4.4.2 <i>Storing Results and Metadata</i>	61
4.4.3 <i>Data Tracking and Analysis</i>	62
4.5 DEPLOYMENT	62
4.5.1 <i>Gradio Deployment</i>	62
4.5.2 <i>Local Deployment</i>	64
4.5.3 <i>Continuous Integration</i>	65
CHAPTER V	67
EXPERIMENTATION AND RESULTS.....	67
5.1. MODEL TRAINING AND TESTING SET	67
5.1.1. <i>Software Configuration</i>	67

5.1.2. Dataset Split (Training, Validation, Testing)	67
5.1.3. Preprocessing (MFCC Extraction, Normalization)	68
5.1.4. Hyperparameters Tuning (Batch Size, Learning Rate, Epochs)	69
5.1.5. Early Stopping and Regularization Techniques	69
5.2. EXPERIMENTAL RESULTS	70
5.2.1. Overall Model Accuracy (Train, Validation, Test)	70
5.2.2. Confusion Matrix Analysis	71
5.2.3. Precision, Recall, and F1-ScoreDatasets	72
5.2.3.1. Precision	72
5.2.3.2. Recall	72
5.2.3.3. F-1 Score	73
5.2.4. Receiver Operating Characteristic (ROC) Curve Analysis	73
5.2.5. Model Loss and Accuracy Curves	74
CHAPTER VI.....	77
CONCLUSION AND FUTURE WORK	77
6.1. CONCLUSION	77
6.2. CONTRIBUTION TO THE FIELD	78
6.3. FUTURE WORK AND RECOMMENDATION	79
REFERENCES.....	83

LIST OF FIGURES

Figure 1 Neural Network	21
Figure 2 Recurrent Neural Network (RNN) Cell	25
Figure 3 Long Short-Term Memory (LSTM) Cell	29
Figure 4 Forget Gate in an LSTM Cell	30
Figure 5 Input Gate in an LSTM Cell	31
Figure 6 Output Gate in an LSTM Cell	32
Figure 7 Gated Recurrent Unit (GRU) Cell	35
Figure 8 Update Gate in a Gated Recurrent Unit (GRU)	35
Figure 9 Reset Gate in a Gated Recurrent Unit (GRU)	36
Figure 10 CRNN Process	38
Figure 11 Block Diagram	40
Figure 12 Block Diagram of MFCC	44
Figure 13 Proposed CRNN model	45
Figure 14 React Frontend Application	52
Figure 15 React Speech Report	53
Figure 16 Audio Playing	54
Figure 17 User Form	55
Figure 18 Dashboard	56
Figure 19 Backend Request	57
Figure 20 API	58
Figure 21 Model Interference	60
Figure 22 Gradio WebApp for Model Deployment	63
Figure 23 Gradio Chatbot	63
Figure 24 Dataset Split for Training , Validation , Testing	67
Figure 25 Confusion Matrix	71
Figure 26 ROC curve	73
Figure 27 Model Loss and Accuracy Curves	74

ABBREVIATIONS

AI – Artificial Intelligence

CRNN – Convolutional Recurrent Neural Network

CNN – Convolutional Neural Network

RNN – Recurrent Neural Network

GRU – Gated Recurrent Unit

LSTM – Long Short-Term Memory

MFCC – Mel-Frequency Cepstral Coefficients

ROC – Receiver Operating Characteristic

AUC – Area Under the Curve

TPR – True Positive Rate

FPR – False Positive Rate

F1-Score – F1 Measure (Harmonic mean of Precision and Recall)

TP – True Positive

FP – False Positive

FN – False Negative

TN – True Negative

SVM – Support Vector Machine

MLP – Multilayer Perceptron

STFT – Short-Time Fourier Transform

API – Application Programming Interface

MSE – Mean Squared Error

RMSE – Root Mean Squared Error

ReLU – Rectified Linear Unit

SGD – Stochastic Gradient Descent

NLP – Natural Language Processing

CHAPTER I

INTRODUCTION

1.1. BACKGROUND

Early detection of speech and language development problems in children is crucial for their cognitive and social growth, and identifying issues as soon as possible is essential. Existing approaches of accessing speech development assessment are generally applied by hand, time-consuming, and require the overall interpretation by a speech-language pathologist. While these methods are effective, they are time-consuming and can also be inconsistent due to subjective opinions. The advancement of Deep Learning and AI automatically opens an avenue towards speech assessment automation and increasing its accuracy to render it more accessible and confident. This thesis, through the advanced deep learning models in speech development analysis, takes a deeper look into using the CRNN model, which integrates CNN and RNN, applied in classifying the speech development in children between 3 and 12 years. This is demonstrated by the collection of an extensive database on speech by children; the latter would then be preprocessed in minute detail with feature extraction, including MFCC and LPC. It helps the models to calculate the patterns over the period and predict the corresponding developmental phase as they are components of the acoustic features in the speech (Modular features captures time spaces in the data with CRNN and CNN networks.)

This is well suited for models like LSTM since this type of combined RNN has long term dependencies that are appropriate for such sequential information such as speech. Another model is the GRU, which is a computationally efficient alternative with largely unrivaled predictive power. Several metrics, including accuracy, precision, recall, and F1 score, have been used to train both models.

From these results, it is obvious that CRNN outperforms the most well-known methodologies for speech assessment considerably. Although the GRU models were much faster in training, LSTMs had marginally better accuracy, making them highly effective for this task.

As much as the results are encouraging, a couple of challenges are inherent in the study. First is that the realization of the best model performance is hindered by a lack of access to high-quality, diverse datasets. The second is related to the fact that the training of deep learning models on large-size speech data has a computationally intensive focus and the task, respectively, requires appropriate infrastructure and corresponding knowledge. Last, ethical concerns include data protection, and the consideration of child's speech and ethicalities should be managed appropriately to attain and meet legal and ethical standards. The proposed AI-based platform is a significant enhancement in the situation, where real-time solutions to speech analysis employ automation. The system provides unbiased and valid assessment—in this respect, a very good tool for parents, teachers, and speech-language pathologists since this will help with earlier detection and resolution of speech disorders. The platform not only streamlines the process by which the speech assessments are availed, but it also reduces the inherent subjectivity that characterizes the manual assessments. This provides one of the key foundations for incorporating AI into the assessment of pediatric speech, as it makes up for deficiencies seen in the traditional methods. The results underlined the potential of AI-powered solutions for enhancing reliability, scalability, and accessibility for early identification and intervention programs. Future studies shall be directed to increase the diversity of datasets and enhancement of interpretability for further optimization of clinical performance in AI models.

1.2. PROBLEM STATEMENT

The traditional detection of speech development disorders in children has been done through the assessment by a speech-language pathologist, which often requires very subjective manual tasks, is time-consuming, and easily inconsistent. These set up certain limitations to the provision of timely and accurate diagnosis and may delay the essential interventions in the affected children. There is, therefore, an increasing call to circumvent such difficulties for an immediate automated, objective, and scalable solution that suits high accuracy in speech development issue detection.

This project addresses this need by developing an AI-driven solution using **Convolutional Recurrent Neural Networks (CRNN)**, which integrate **Recurrent Neural Network (RNN)** and **Convolutional Neural Network (CNN)**. The model leverages these advanced deep learning techniques to classify speech development stages and identify potential speech disorders in children aged 3 to 12. In such a way, the system will improve accuracy, improve the speed, and

make the early diagnosis more reliable by integrating temporal and spatial features of speech into one system; hence, it offers a scalable alternative to traditional techniques. This system is envisioned to become an automated tool that can aid clinicians, educators, and even parents in delivering timely interventions to children with speech development issues, which is expected to improve outcomes.

1.3. PROPOSED SOLUTION

The proposed solution here, therefore, is the creation of an AI system utilizing deep learning to identify the stages, and any disorders, in the child's speech. This work employs a design based on the CRNN structure, which refers to a CNN incorporated with an RNN. The proposed system employs the adopted hybrid model to understand spoken language of children, extract the features of children speech and, finally, classify them by developmental stages or to signal the presence of such disorders as stuttering or stammering in case they exist.

The system begins with extensive data preprocessing, where raw speech audio files are processed using techniques such as Mel-Frequency Cepstral Coefficients (MFCCs) and Linear Predictive Coding (LPC) to extract key acoustic features. These features represent the frequency and temporal characteristics of the speech, which are critical for understanding the nuanced patterns of speech development. After feature extraction, the data undergoes normalization to ensure consistency in scale, and data augmentation techniques like pitch shifting and time-stretching are applied to increase the robustness of the model by providing a more diverse set of training samples. This preprocessing pipeline ensures that the system can handle variations in speech quality and recording environments, making it adaptable to real-world conditions.

The overall structure of CRNN model comprises of multiple convolution layers over filters that help in extracting local patterns from spectrograms and pooling layers that in decreasing the feature length without losing important features. These features are then forwarded for RNN layers which work in identification of sequential patterns in the speech data, to help the system about how the sounds change over time. CNN and RNN integration enable the model to work both in spatial and temporal domains and thereby makes it suitable for identifying developmental patterns of speech and speech abnormalities.

The model training period involves preparation of sets which include training, validation and the test set. To enhance the training process the Adam optimizer is applied which allows learning with minimal categorical cross-entropy loss with corrected weights. To tackle this problem, two classes of methods are used, early stopping and dropout and therefore the model is able to generalize well from the training set to other unseen sets. While the layers of the neural network as an equation are hyperparameters, other hyperparameters such as the learning rate, the batch size, are carefully tuned using both grid search and random search to achieve the best model performance.

Once trained, the model is evaluated using key metrics such as accuracy, precision, recall, and the F1-score, providing a comprehensive understanding of the system's ability to distinguish between normal and disordered speech. Additionally, a confusion matrix is used to identify areas where the model may need further improvement, such as differentiating between similar developmental stages or avoiding false positives in disorder detection.

The final system is designed to function in real-time, with the trained model deployed using tools like TensorFlow Serving to enable instant feedback on speech assessments. This real-time capability makes the system a practical tool for speech-language pathologists, educators, and parents, providing immediate and accurate insights into a child's speech development. By automating the assessment process, this solution reduces the reliance on manual evaluations, making speech development assessments more scalable, efficient, and accessible, especially in areas where professional resources are limited.

1.4. SIGNIFICANCE

To identify the speech development disorders in the children at a very young age brings about a lot of benefits as far as the child's cognitive, social and academic development. Evaluations of speech data that cover pre-analysis done manually by experts tend to be less accurate, time-consuming and expensive. To conclude, we propose the use of deep learning, specifically Convolutional Recurrent Neural Networks (CRNN) to automate the assessment process to be more efficient and consistent. This work advances the knowledge in the field of speech pathology because it employs RNN and CNN architecture exclusively to offer an effective, Deep learning-based solution for early and accurate diagnosis of speech impairment in children. This technology can be of great help to speech apt professionals, teachers, and caregivers since they contain real-time feedback which may be useful for early intercession in order to set the appropriate

developmental results. In addition, the study supports the advancement of the use of AI in healthcare and gives a model of operation that is effective for future expansion of pediatric speech therapy.

1.5. RESEARCH OBJECTIVE

- The To create a CRNN-based mechanism for automated classification of Speech development stages in children ranging between 3-12 years.
- To intelligently implement a processing chain, in order to collect MFCC and other related features derived from children's speech samples.
- To measure the effectiveness of the proposed CRNN model by means of metrics including, accuracy, precision, recall, F1-score and by drawing ROC curve for high reliability.
- To improve early identification of speech Impairment, developing solutions that are real time, manageable and easy to access for teachers, speech Therapists, and Care givers.
- for the purpose of promoting the achievement of the United Nations' SDGs by enhancing the delivery of education and healthcare services through The use of Artificial Intelligence.

1.6. RESEARCH QUESTIONS

To what extent does the architecture of the proposed CRNN model enable the differentiation between stages of speech development in children between the ages of 3 and 12?

Which preprocessing features are most important for improving the CRNN model's performance?

What is the comparison of the proposed method to the prior conventional methods in aspects of accuracy, efficiency and reliability for analysis of speech development?

Can the system provide real-time feedback with sufficient reliability for practical use in diverse environments?

What are the ethical considerations in using AI for assessing children's speech development, and how can they be addressed?

1.7. ALIGNMENT WITH SUSTAINABLE DEVELOPMENT GOALS

The research project aligns with several Sustainable Development Goals (SDGs) set by the United Nations. Here's how the project contributes to these goals:

SDG 3: Good Health and Well-being

- **Target 3.4:** Exclude premature mortality from non-communicable diseases by preventing as well as treating and advancing mental health and well-being.

- **Alignment:** In particular, the goal of the project is to identify the risk of speech development disorders at the early stage, so it is vital for the children's subsequent cognitive and social development. It is a medical adage that importantly, early intervention is likely to produce better results in terms of well-being.

- **Target 3.8:** Achieve universal health coverage, including financial risk protection, access to quality essential health-care services, and access to safe, effective, quality, and affordable essential medicines and vaccines for all.

- **Alignment:** In addition, the project of developing the automated system of assessment of speech development can be useful in increasing availability and availability of healthcare services in the regions where the qualified staff is scarce.

SDG 4: Quality Education

- **Target 4.2:** Ensure that all girls and boys have access to quality early childhood development, care, and preprimary education so that they are ready for primary education.

- **Alignment:** The project can help educators to detect delay in child's speech and as a result take the necessary measures to help the kid be ready for the next step – primary education.

SDG 9: Industry, Innovation, and Infrastructure

- **Target 9.5:** Enhance scientific research, upgrade the technological capabilities of industrial sectors in all countries, in particular developing countries, including, by 2030, encouraging innovation and substantially increasing the number of research and development workers per 1 million people and public and private research and development spending.

- **Alignment:** The project actively applies profound human and artificial intelligence technologies of deep learning and innovative tech in responding to a vital health care challenge and thus contributing to the advancement of the technological qualitative characteristics and novation's in health care.

CHAPTER II

LITERATURE REVIEW

2.1. INTRODUCTION

The early detection and assessment of speech development disorders in children has long been a significant challenge in pediatric healthcare and education. Direct observation is conventional evaluation methods that entail the use of professional judgment from speech-language pathologists; they are valid but restricted by issues like bias, time-consuming and variability. The new sophisticated technique provided by AI and an approach called deep learning can become the best solution for this process since it is free from human subjectivity and can be scaled easily. Previously, the assessments were observational checklists and tests which, although offered rich information, demanded much of the expertise and time. These were time consuming and in addition Human judgment being subjective, leads to variability in diagnosis which is likely to bring variation in the early identification of the speech development disorders.

More and more children are now experiencing developmental speech disorders and there is this ever growing demand for early and accurate analysis that can be made quickly and without much human intervention. Among of the deep learning models, Recurrent Neural Networks including Long Short-Term Memory and Gated Recurrent Units have been successfully used for analyzing sequences of signals, as speech. Due to their ability of capturing temporal dependencies and patterns from the speech data they are especially useful in speech recognition and classification and for diagnosing developmental disorders. Interestingly, concerning the accuracy and robustness of models, LSTM and GRU models are significantly superior to the traditional machine learning clustering methods and provide an empirical approach to assess speech development.

Several works have investigated the use of LSTM and GRU networks in different speech—related problems. For example, one of the studies on Bengali speech recognition employing double-layered LSTM-RNN showed a rather improved performance over the conventional techniques, in

particular, the feature of LSTMs in learning complexity of the speech data with temporal structure. Yet one of the recent comparative studies of GRU and LSTM-based Automatic Speech Recognition show the superiority of both models with a slight edge in accuracy for LSTMs against the relatively faster training times offered by the GRU models. These results support the future developments of both models in expanding speech assessment possibilities, providing accurate and effective means for speech development evaluation.

Nevertheless, there are several problems still persisting in the field of speech assessment assisted by deep learning. One important question is data demands – often, vast and heterogeneous datasets are wanted because the effectiveness of AI depends on the training data. Still, one indexes the purchase of high quality, labeled speech data, especially children as a major challenge response. In addition, since most deep learning models, including LSTM and GRU, consume large amounts of computer power when learning new information, their training can be difficult. Data privacy and security are also important issues besides concern for protection of privacy and dignity of children since data being collected may be sensitive in nature. The issues of consent, data processing and potential small risks of data leak must be addressed to make AI deployment in this area as ethical as possible.

There is another crucial issue, which concerns the possibility of interpreting the results, provided by AI models. However, LSTM and GRU models deliver high accuracy while there are drawbacks that the model's decision-making process in real world and black box in nature. This lack of transparency can be problematic for clinicians and educators who may want easily interpretable explanations about how a diagnosis or prediction is made. Model interpretability remains critical to the use of AI-based tools to avoid incorrect outcomes and misuse of such tools in clinical or educational expectations.

This literature review was conducted with the purpose to perform the analysis of the current state of knowledge on the application of AI approaches for the assessment of speech development. It will include an outline of deep learning from the ground up together with key emphasis on LSTM and GRU approaches and possible new research into methodologies used for feature engineering and data preprocessing. Some of these methods include Mel Frequency Cepstral Coefficients

(MFCCs) and Linear Predictive Coding (LPC) which are very important in the feature extraction process that is used for converting raw speech data into features that can be used by the model. Also, the review will outline current approaches to speech evaluation, difficulties that these approaches face, and how AI solutions can help eliminate those challenges, in order to suggest more accurate, efficient, and unbiased evaluation methodologies. This research focuses on the challenges and seeks to build on the strengths of deep learning to contribute to efforts of developing the AI solutions to revolutionize the early identification and response to speech development disorders.

2.2. OVERVIEW

Deep learning, a key area within machine learning, has transformed various fields by enabling systems to learn from large datasets and make intelligent decisions. Closely related to this advancement are varieties of neural networks, a multilayer pattern of nodes with interconnection resembling the structure of the human brain. Such networks are able to learn patterns and even they are adaptive with high level training. In the context of this project, neural nets, especially sequential ones for modelling speech, contribute significant tools for the automatization of speech development assessment for children. These networks, when trained to make use of the spatial and the temporal characteristics of the speech signal, can detect speech disorders with high accuracy, efficiency and early enough to allow clinicians and educators to make suitable interventions.

2.3. NEURAL NETWORK

A neural network is a machine learning program, or model, that makes decisions in a manner similar to the human brain, by using processes that mimic the way biological neurons work together to identify phenomena, weigh options and arrive at conclusions. Every neural network consists of layers of nodes, or artificial neurons—an input layer, one or more hidden layers, and an output layer. Every node is connected with others, and is inclusive of weight and threshold too. If the output of any individual node is greater than the threshold value that is decided then that node is excited and gives its outputs to the next layer of the network. Otherwise, no values are transmitted to the next layer within the networked system.

Neural networks trained with new data and are capable of improving their performance once they have been trained with new data. Neural networks can also be referred to as artificial neural

networks (ANN), or simulated neural networks (SNN). They belong to the family of both machine learning and at the same time under the deep learning architecture.

2.3.1. How do Neural Network Works

Neural networks are composed of a collection of nodes. The nodes are spread out across at least three layers. The three layers are:

1. An input layer
2. A hidden layer
3. An output layer

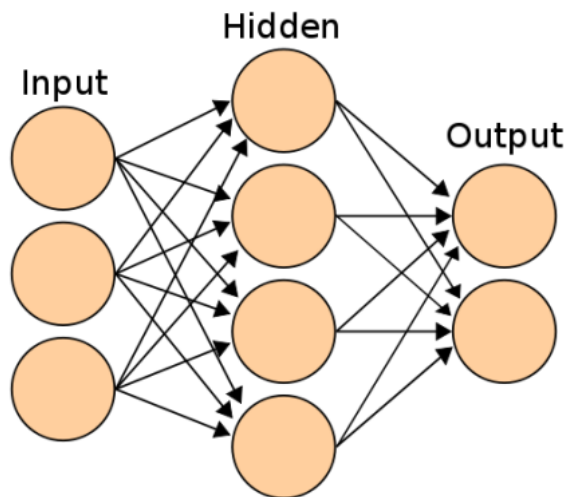


Figure 1 Neural Network

These three layers are the minimum. Neural networks can have more than one hidden layer, in addition to the input layer and output layer.

No matter which layer it is part of, each node performs some sort of processing task or function on whatever input it receives from the previous node (or from the input layer). In other words, each node holds a mathematical formula where all the variable can be weighted uniquely. In order for the node to pass data to the next layers of the neural network there is a certain threshold that when the output of the math formula applied to that input surpasses, then data is passed onto the next layer. If the output is still below the threshold no input passes through to the next layer.

Think of each individual node as its own linear regression model, composed of input data, weights, a bias (or threshold), and an output. Once an input layer is determined, weights are assigned. These

weights make it possible to quantify the significance of the particular variable with the magnitude of the weights used in the computation of the output greater than other inputs. All inputs are then scaled by their weight coefficients and the quantities so obtained are added. Subsequently, the output formed is then passed to an activation function, from which the output is obtained. If that output goes above a certain limit; it triggers the node known in the basic form as ‘fires’ this will relay the information to the next layer in the network. This leads to the creation of a situation whereby what is outputted by one node is inputted by the next node. Such a flow of data from one layer to another layer in this neural network makes this a feedforward network.

In the end, it is aimed to reduce the magnitude of our cost function which gives us an optimal fit for any observation we have. During the model training, when weights and bias are get updated, cost function and reinforcement learning is used to get the local minimum which is the point of convergence. The technique by which the weights are updated is known as the gradient descent and with this the model is capable of identifying the direction to undertake in order to decrease on the error (or to minimize the cost function).

2.4. TYPES OF NEURAL NETWORKS

Neural networks are versatile models that can be classified into various types based on their structure and application, each designed to handle specific tasks. In our project we used multiple types of neural networks with greater focus on one type of neural network namely Convolutional Recurrent Neural Network (CRNN) which has attributes of both CNN and RNN as it is particularly suitable for the speech data.

Feedforward Networks: A feedforward neural network is the basic form of neural architectures where data pass through only in one direction without any backward or feedback loops. This type of network is applied to such tasks as, pattern recognition and regression type of problems but cannot be used for such sequence problems as speech where sequence or temporal information is important.

Multilayer Perceptron (MLP): The MLP is just an advanced extension of the feedforward network which includes an input layer one or multiple hidden layers and an output layer. The MLPs take the non-linear activation functions in order to express the relationships. They can

perform classification and regression analysis and similar to feedforward networks they over temporal strategies or in other words, they are not as effective for time sensitive data inputs such as speech signals.

Convolutional Neural Network (CNN): CNNs are mainly applied to work on the rectangular-shaped data sets, e.g. images or spectrograms. In our work, CNN layer is used to extract the feature from spectrograms of speech which describe the frequencies and time samples of the audio data. CNNs are also capable of learning spatial features for example changes in pitch or tone over time. This make them invaluable in our speech analysis pipeline because they can index the local neighborhood of such features which are needed to characterize patterns related to speech development or disorders.

Recurrent Neural Network (RNN): RNNs are planned to route series data where the previous input impacts the current output. They are very much useful in areas such as speech processing, since the order of the sounds actually makes a difference in the processing. RNN contains feedback connections, enabling them to remember the earlier inputs thus making them suitable in handling time series inputs. Thus, the primary drawback of traditional RNNs is the exposure to long-term dependency, due to which state-of-art models such as LSTMs and GRU were developed.

Long Short-Term Memory (LSTM): LSTMs are special forms of RNNs developed specifically to solve an annoying problem known as vanishing gradient issue that could arise when training long sequences. LSTMs employ memory cells and gating mechanisms to know what to retain and what to forget and due to this makes it useful in speech analysis, where there are long term dependency. In our project, LSTMs are important in establishing elaborate pattern flow inherent in children's speech development, through the ability to store information required for future classification of distinct patterns of speech disorders, for an extended period.

Gated Recurrent Units (GRU): Compared to LSTMs GRUs are also meant to model temporal dependencies in sequential data yet with less complex structure. In general, GRUs require less gates than LSTMs making them to solve computational problems in less time hence faster training time. Our results also demonstrate that while GRUs are faster than LSTMs, though not as accurate,

they could provide better real-time speech analysis as compared to LSTMs and are better at detecting more complex and sophisticated speech patterns.

Convolutional Recurrent Neural Network (CRNN): CRNN is the central foundation of our task, which adopts both the convolutional and recurrent neural networks. CNN layers in this case extracts spatial features from the spectrograms of speech in specific regions such as pitches and tones while the RNN layers (LSTM or GRU) captures the temporal characteristics of the data informing the model how speech changes over time. When the data and or characteristics analyzed by the model involve spatial and temporal elements this hybrid architecture is particularly powerful as shown by the case of speech development assessment which depends on temporal sequences of sounds and spatial characteristics of the sounds. In our work on the JS DR project, the CRNN model has given remarkable performance for synthesizing children speech, understanding milestones of child speech development and diagnosing a speech pathology with greater accuracy.

2.5. RECURRENT NEURAL NETWORK (RNN)

A recurrent neural network (RNN) is a deep learning model that is trained to Sequential data is data. In traditional neural networks, all the inputs and outputs are independent of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is process and convert a sequential data input into a specific sequential data output. a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its Hidden state, which remembers some information about a sequence. The state is also referred to as Memory State since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.

These deep learning algorithms are commonly used for ordinal or temporal problems, such as language translation, natural language processing (nlp), speech recognition, and image captioning; they are incorporated into popular applications such as Siri, voice search, and Google Translate. Explaining similarly with feedforward and Convolutional Neural Networks, Recurrent neural networks learn from training data. They are characterized by ‘memory’ since the current input and

output depend of information coming from the previous inputs-outputs. The inputs and outputs of the treatment of deep neural networks are random while for recurrent neural networks; successive outputs are dependent on the previous items of the series. In the same way that future events would also be useful in predicting the outcome of a particular sequence, unidirectional recurrent neural networks lack the ability to consider these events in the results. Vanilla RNN or simple RNN works in sequence processing spreading the hidden state that incorporates information about the inputs in the current sequence. At each of the step l , there are some calculations that the RNN undertakes the final information the RNN provides is an output. This means to support classification tasks, a single output is expected. However, for text generation to be performed based on the previously generated word, an output has to be produced at every time step.

2.5.1. Working of RNN

In Recurrent Neural Networks (RNNs) information passes through the network and then loops back through the middle hidden layer to give the network a form of memory, known as the input layer, accepts the first data and after its processing forward it to the middle layer. This middle layer, namely h , can have more than one hidden layers which can consist of more than one activation function, weight and bias. For these layers assist in converting the input data into a suitable format that can be processed within the network and make sensible predictions.

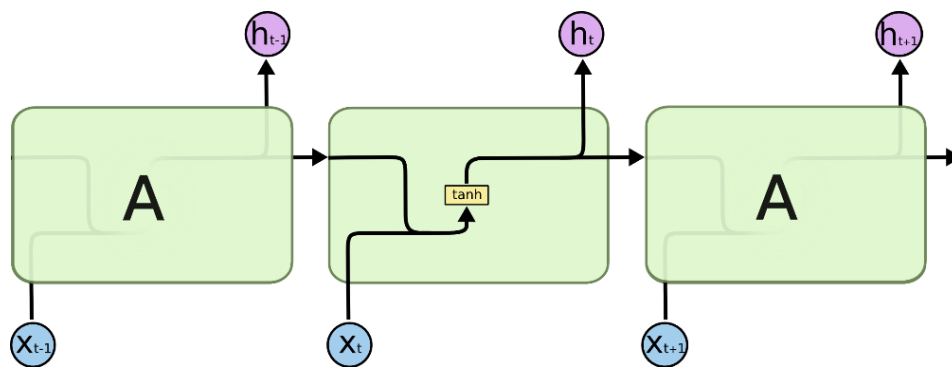


Figure 2 Recurrent Neural Network (RNN) Cell

Unlike the usual neural network, RNN does not have each layer functioning on their own without the knowledge of past data, as RNN has a looping systems. If the parameters of the network do not depend on the previous layers output, thus the network has no memory, a feed forward network can be used. Yet for those tasks, an RNN is applicable since it is required that the network remembers some information from the past and apply it to the current situation.

In an RNN the network structure is intended to mean that all activation functions, weights and biases are made equivalent across the time step of a sequence. This means that at every time step, the same number of parameters is used for all the hidden layers in adjustment of weights or in the control of state transitions. This forms a recurrent structure in which instead of creating many hidden layers it has one hidden layer which has a cyclic nature of as many time steps as required. This layer, in their case, is unrolled but changes after each loop upon receiving an input as well as the memory of the previous timestep output.

Through this looping mechanism, the RNN can process a sequence of data so easily because the same hidden layer is used, and thus it can have content from previous elements. This capability is important for applications where data is temporal in nature, like speech recognition where each language sound depends on previous sounds. Through this looping of the same layer, RNNs are able to hold the former steps into their memory and fine-tune subsequent predictions relying on context elaborated by previous inputs.

2.5.2. Limitation of RNN

Vanishing Gradient Problem:

During the training of our developing speech assessment model we observed one of the major drawbacks of RNNs which is the vanishing gradient problem. This problem occurs when the RNN is applied to segments with a large number of sequences: for instance, sequences composed of long stretches of audio children's speech. While back propagating they used for adjusting weights of the network, the gradient becomes very small while it propagates back through the layers and time stamp. Hence it is almost impossible for the network to update its weights efficiently. This leads to difficulty in learning long-term dependencies, which are crucial for our task. In speech data, recognizing patterns that span across long time periods is essential for accurately identifying speech disorders. However, the vanishing gradient problem limits the network's ability to retain information from earlier parts of the sequence, affecting the model's accuracy in detecting subtle, long-term patterns in speech development.

Exploding Gradients:

Although there are cases when gradients are too small due to the vanishing gradient problem, we encountered cases also with exploding gradients. This arises particularly at a time the gradients are huge during backpropagation especially when the network is dealing in a sequence acknowledgment. The large gradient lead to oscillation during the training as the weights jump all over the place; this is a form of instability. This may lead the model to develop what is called an emergency: meaning the model is not able to learn from the data or worse the network parameters become too large, and the training process becomes unstable. Faced with the problem of training instability that can manifest itself during the training of models with many layers, in our project, while processing long sequences of speech data, we had to navigate around the problem of exploding gradients.

Limited Memory:

The other major drawback of RNNs which we also come across during our FYP is that they have short-term memory. The problem with a regular RNN is that the network is dependent on the hidden state which carries forward information from previous time steps. This state is however of small capacity which poses a drawback to the network as it cannot hold important information over long sequences. In speech assessment, for instance, the model needs to remember details that span a larger time horizon, for example, decline in speech growth. However, typical RNNs have problems in preserving information from earlier segments of a sequence, and hence they suffer from difficulties in the learning from long-term relationships. This limitation was problematic in our work because speech disorders can often be defined by long term temporal patterns and the relatively short memory of RNNs put limits to the model.

Difficulty in Training:

Because of the problems with vanishing and exploding gradients and the general lack of memory in a traditional RNN, it was difficult to train these networks for large tasks when the sequences were long. In our FYP we chose the model to analyze long speech recordings and because of these

limitations the training process took longer and it was difficult to adjust the network for the best results. The training problem emerged when trying to capture finer details of the qualitative differences distinguishable in the learner's speech, which meant the network had to possess knowledge from the short-term and long-term portions of the sequence. This caused the RNN to perform poorly on unseen data and was a major factor preventing the accurate identification of the stages of speech development or the presence of disorders. For this reason, LSTM and GRU type models were developed to address these limitations not aimed well.

2.6. LONG SHORT-TERM MEMORY (LSTM)

Long Short-Term Memory (LSTM) is a special type of Artificial Recurrent Neural Network (RNN) used deep learning techniques. While most feedforward neural networks used in other machine learning applications are basically feedforward networks for brief fixed-size data, LSTMs contain feedback connections which enable them to utilize sequential dependencies present in sequences of data. LSTM is intended to be able to solve the problem with vanishing or exploding gradients when training regular RNNs on sequential data. Thus, they are most suitable for application in problems that require processing sequential data, such as Text to Speech and vice versa-TTS, Language Translation, Speech recognition and (Time series analysis and forecasting and so on.

LSTM networks add memory cells, which is able to store information over the complete sequences. Each memory cell has three main components: We got to know it as an input gate, a forget gate, and an output gate. They assist in managing the traffic within and out of the memory cell Most common gates are;

The control signal of the input gate influences to which extent the new input signal should be written to the memory cell. The formula takes current input and previous hidden state as input and gives us the normalized output for each memory cell from 0 to 1.

The forget gate decides what has to be erased or removed from the memory cell. This takes the current input and the previous hidden state as input and yields an output vector between 0 and 1 in dimension of memory cell. A value of 0 reckons the information as non-significant and a value of 1 reckons it as significant. The output gate regulates the amount of the memory cell's content

which will be utilised to calculate the hidden state. Actually, it consumes the current input that is actually an epoch and the previous hidden state to output the value between 0 and 1 for every element of a memory cell.

These gates make it possible for LSTM networks to regulate memory writing, memory update and memory retrieval over sequences. Which makes them rather appropriate for modeling long term dependencies tasks such as speech recognition, language translation or sentiment analysis.

2.6.1. Logic Behind LSTM

The central role of an LSTM model is held by a memory cell known as a ‘cell state’ that maintains its state over time. The cell state is the horizontal line that runs through the top of the below diagram.

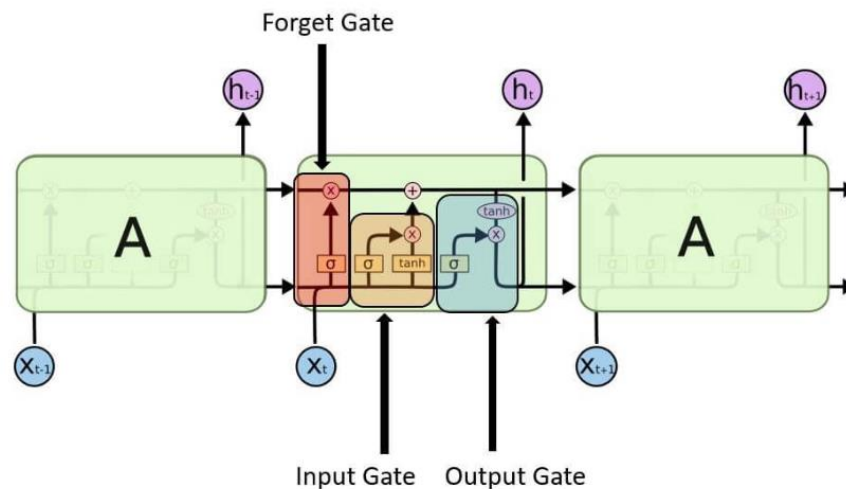


Figure 3 Long Short-Term Memory (LSTM) Cell

It can be visualized as a conveyor belt through which information just flows, unchanged. Information can be added to or removed from the cell state in LSTM and is regulated by gates. These gates optionally let the information flow in and out of the cell. It contains a pointwise multiplication operation and a sigmoid neural net layer that assist the mechanism. The sigmoid layer gives out numbers between zero and one, where zero means ‘nothing should be let through’, and one means ‘everything should be let through’.

2.6.2. Architecture of LSTM

The LSTM architecture involves the memory cell which is controlled by three gates: In the long short-term memory network, there are three types of access state; the input gate, the forget gate, and the output gate. There are three gates that control the input, the removal and output of information to and from this memory cell and each part serves a singular role. The input gate determines the amounts and type of information that is brought into the memory cell.

The forget gate determines which data is to be erased in the memory cell. The output gate regulates which data is to be read out from the memory cell.

2.6.2.1. Forget Gate

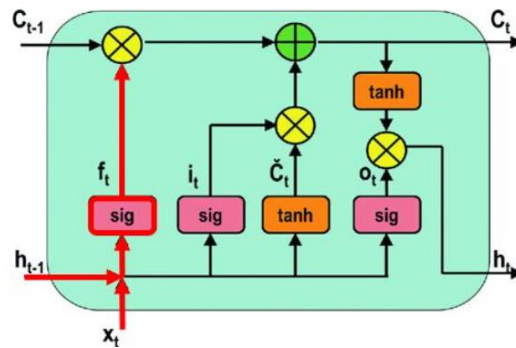


Figure 4 Forget Gate in an LSTM Cell

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Where,

t = timestep

f_t = forget gate at t

x_t = input

h_{t-1} = Previous hidden state

W_f = Weight matrix between forget gate and input gate

b_f = connection bias at t

The forget gate decides which information needs attention and which can be ignored. The information from the current input x_t and hidden state h_{t-1} are passed through the sigmoid function. Sigmoid generates values between 0 and 1. It concludes whether the part of the old output is necessary (by giving the output closer to 1). This value of f_t will later be used by the cell for point-by-point multiplication.

2.6.2.2. Input Gate

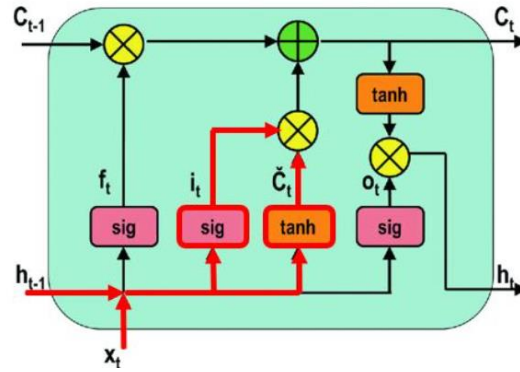


Figure 5 Input Gate in an LSTM Cell

$$i_t = \sigma(W_i \cdot [h_t, x_t] + b_i)$$

$$C_t = \tanh(W_C \cdot [h_t, x_t] + b_C)$$

Where,

$t = \text{timestep}$

$i_t = \text{input gate at } t$

W_i

= Weight matrix which interconnects sigmoid operator between input gate and output gate

$b_i = \text{bias vector at } t$

$C_t = \text{value generated by tanh}$

$W_C = \text{Matrix weight between tanh operator and OSI of cell state information}$

$b_C = \text{bias vector at w.r.t } W_C$

The input gate performs the following operations to update the cell status. First, the current state x_t and previously hidden state $h_{(t-1)}$ are passed into the second sigmoid function. The values are reduced from the range between 0 (important) to 1 (not-important). Finally, the same information of the hidden state and current state will pass through the tanh function. In order to use this mechanism for controlling the network, the tanh operator will input into the system a vector ($C_{(\sim t)}$) that contains all the values that range from -1 to 1. The values that come out from the activation functions are in a form suitable for point by point multiplication.

2.6.2.3. Output Gate

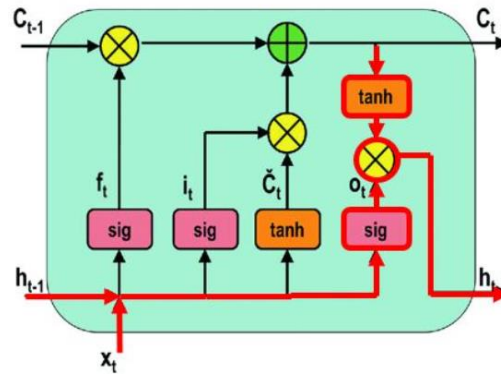


Figure 6 Output Gate in an LSTM Cell

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$C_t = o_t * \tanh(C_t)$$

Where,

t = timestep

f_t = forget gate at t

x_t = input

h_{t-1} = Previous hidden state

W_f = Weight matrix of forget gate with hidden state

b_t = connection bias at t

The choice of the next value of the hidden state depends on the output gate as well. It has stored information on previous inputs of this state. First, these current state value and previous hidden state value are passed to the third sigmoid function. After that all the cell state which is generated is passed through a tanh function and then we get the new cell state. Even these two outputs are scaled up, point by point. According to the final value the network decides what information the hidden state should contain. There is another hidden state which is used for making predictions. Last of all, the new cell state and new hidden state are transferred to the next time step or to other deeper layers in recurrent networks.

Finally, the forget gate provides information on which data from previous steps are relevant. The input gate determines from which components in the current step can new information be incorporated, the output gates determine the next hidden step.

2.7. GATED RECURRENT UNIT (GRU)

The Gated Recurrent Unit (GRU) refers to a specific type of recurrent neural network architecture that has been specially developed to deal with sequential data while evading some of the difficulties inherent to the traditional RNN type of model. Like LSTM networks, GRUs are good at identifying long-term dependencies within patterns and data and are more efficient in training than LSTM, and are less computationally intensive. Traffic through the network is controlled by two gates—update gate and reset gate in the case of the modified GRUs. The update gate defines how much of the past information should be propagated into the future and the reset gate defines how much of the past information should be ejected. By contrast, GRUs eliminate the requirement of having distinct memory cells as seen in LSTMs, making the overall framework less complex, hence lesser parameters. Nevertheless, it has been demonstrated that GRUs are as effective as LSTMs on many tasks, much more when dealing with sequential data, as in speech recognition, natural language processing, or time series forecasting. Compared to vanilla RNNs, GRUs are easier to compute and perform well when computation speed is a requirement but the model still needs to capture useful patterns in the sequences it works on.

2.7.1. How GRU's work

While GRUs also have feedback connections, they also have gates that let them learn long-term memories that are stored in a stream of data. Two internal gates specifically, Reset Gate and the Update Gate control the amount of past information GRUs spare and the amount of new information it allows. This helps them to extend the recall of information from previous stages of a sequence – as well as with long sequences compared to the baselines where traditional RNNs face challenges. The reset gate which its value is a float between 0 and 1, determines how much of the past hidden state information is to be forgotten. Think about a value very close to zero which would essentially wipe out almost all the preceding data and on the other extreme, a value very close to one that would recall almost all the data. It excludes irrelevant information that would otherwise create a congested memory space in the network.

As with the reset gate, it takes the current input and previous hidden state but its output also controls how of the processed current input that should be included in the new hidden state. A digit close to 0 makes use of previous state salience, whereas, a number close to 1 makes use of the new data. The reset gate's output is pretty much just “pass-through” for previous hidden state but only acceptable information passed through in the process. This filtered past is then augmented with the processed current input, that has passed through the update gate to form the new candidate for the hidden state.

Last but not the least, a new candidate hidden state is added to the previously existing hidden state with the help of the update gate output. It also helps to reduce the noise in knowledge by removing the old rapidly while retaining the new that is most important. Through reset gate, update gate, and candidate hidden state, GRUs can control the update of their hidden state and capture both short and long term dependencies in a sequence data.. This solves the vanishing gradient problem and enables the model to learn more complex patterns in sequences.

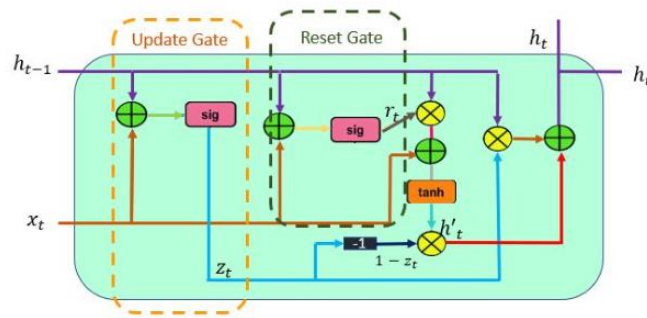


Figure 7 Gated Recurrent Unit (GRU) Cell

2.7.2. Architecture

GRU cell which more or less similar to an LSTM cell or RNN cell. At each timestamp t , it takes an input X_t and the hidden state H_{t-1} from the previous timestamp $t-1$. Later it outputs a new hidden state H_t which again passed to the next timestamp. Now there are primarily two gates in a GRU as opposed to three gates in an LSTM cell. The first gate is the Reset gate and the other one is the update gate.

2.7.2.1. Update Gate

The update gate (z_t) is responsible for determining the amount of previous information (prior time steps) that needs to be passed along the next state. It is an important unit. The below schema shows the arrangement of the update gate.

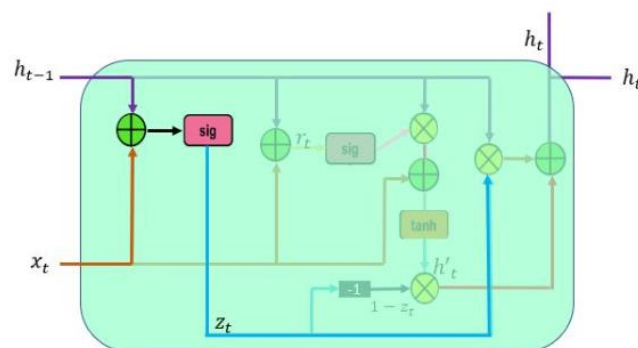


Figure 8 Update Gate in a Gated Recurrent Unit (GRU)

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$$

Where,

t = timestep

$\mathbf{z}_t$ = update gate at t

$\mathbf{x}_t$ = input vector

$\mathbf{h}_{t-1}$ = Previous hidden state

$\mathbf{W}_z$ = Update gate weight matrix

$\mathbf{b}_z$ = connection bias at t

Here, $\mathbf{x}_t$ is the input vector served in the network unit. It is multiplied by its parameter weight $\mathbf{W}_z$ matrices. The $t-1$ in $\mathbf{h}_{(t-1)}$ signifies that it holds the information of the previous unit and it's multiplied by its weight. Next, these parameters are summed up as follows and the values from these parameters are passed through the activation function, which is sigmoid function. In this case sigmoid function would produce limit values within the range of 0 to 1.

2.7.2.2. Reset Gate

The reset gate r_t is used from the model to decide how much of the past information is needed to neglect. The formula is the same as the update gate. There is a difference in their weights and gate usage, which is discussed in the following section. The below schema represents the reset gate.

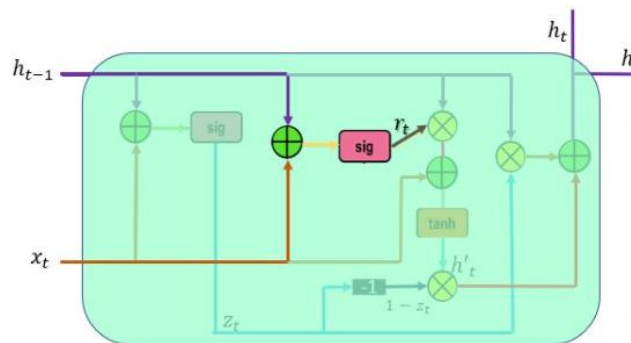


Figure 9 Reset Gate in a Gated Recurrent Unit (GRU)

$$\mathbf{r}_t = \sigma(\mathbf{r} \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$$

Where,

$t = \text{timestep}$

$r_t = \text{Reset gate at } t$

$x_t = \text{input vector}$

$h_{t-1} = \text{Previous hidden state}$

$W_r = \text{Reset gate weight matrix}$

$b_r = \text{connection bias at } t$

The following steps were important for increasing the quality and relevance of the dataset for the next stage of creating the model.

2.8. CONVOLUTIONAL RECURRENT NEURAL NETWORKS (CRNN)

The Convolutional Recurrent Neural Networks (CRNN) are themselves the combination of two distinguished classes of neural networks: convolutional neural networks (CNN) and recurrent neural networks (RNN). It is ideal for converting spatial features from input data and having the ability to convey sequential data most efficiently than other models. This makes CRNNs more useful whenever an analysis of spatial-temporal information is required such as handwriting recognition and speech recognition. Special types of AUTOENCs are CRNNs which include stacked convolutional layer and recurrent layer, they can learn local and global spatial patterns and temporal dependencies. This architecture demonstrated near perfect results in a number of domains and is desirable for the further development of deep learning. .

2.8.1. Architecture of CRNN

In next configuration, the Convolutional Recurrent Neural Network (CRNN) system, has been described and the architecture has been depicted in the following figure, is an effective to learn the spatial and temporal characteristics using one sequence data such as audios CRNNs and RNNs have been combined in this architecture for capturing the spatial and temporal dependencies from a sequential data. This hybrid architecture is especially designed for the speech recognition and classification tasks.

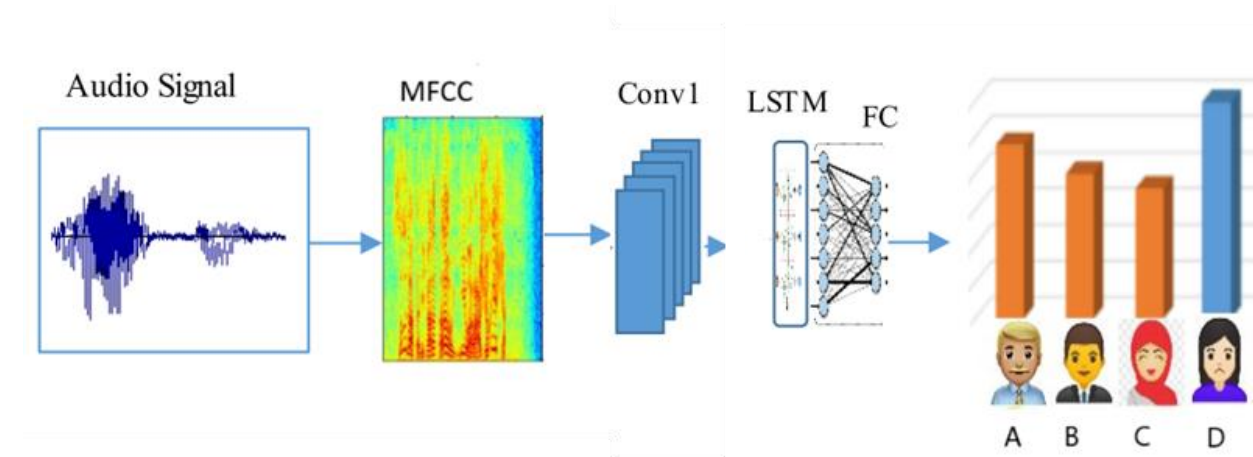


Figure 10 CRNN Process

Input Layer: The input to the model is an input window of 40 MFCC features of the audio signal. MFCCs extract useful information about the raw speech waveform by transforming the waveform into a 2D feature space where the horizontal axis is time and the vertical axis is frequency band. In your model, you have an input layer of size (64x96x1) which would be feature maps that are the raw data for the next stage in your model.

Convolutional Layer (Conv1): In this architecture, we make use of one convolutional layer, the Conv1 layer which possesses a 1D convolution to extract space features from the MFCC input. The Conv1 layer convolves filters across the input in order to find features relevant to the frequency domain. If we were to use multiple convolutional layers, they extract general features which are sufficient for this model instead of compounding the network. When we use the Conv1 layer, the dimensionality of the output is tremendously high; hence, Max Pooling is employed subsequent to Conv1 so as to retain the most important information about the input.

Recurrent Layers (LSTM): After that, the generators of feature maps are put into LSTM layer which is called Long Short Term Memory. The LSTM layer is used here in order to capture temporal dependencies in the data – that is, learn how features change over time. These features make LSTM cell appropriate for the tasks like speech; it keeps memory across the time steps. The LSTM takes an output from the convolutional layer as its input and passes a sequence of feature that describe the temporal characteristics in the speech signal.

Fully Connected Layer (FC): Finally, after the LSTM networks analyze the sequential data, the results are shaped to be fully connected layer ready. The last emergent layer is used to make fully connected layer that maps the high features from the previous layer of convolution and LSTM layers to the output. This layer contracts the feature space and yields the final result for the given classification problem.

Output Layer: The output layer has 5 neurons which correspond to 4 different age ranges chosen for the analysis according to the data set (from 3 to 12 years of age). The final layer of the neural network uses the softmax activation function to generate probability degrees of pupils' belonging to the particular age group where the speech samples must be classified.

CHAPTER III

METHODOLOGY

3.1.OVERVIEW

The methodology for this study is centered around designing and implementing a Convolutional Recurrent Neural Network (CRNN) that integrates CNNs for spatial feature extraction and RNNs for temporal sequence modeling. It comprises data gathering, data preparation, model development, model training, model assessment, and model utilization. The dataset includes only speech samples, with children aged between 3 and 12 years and divided by developmental stages, respectively, and converted into MFCCs format. Preprocessing pipeline makes them more consistent and robust by standardize and increasing the number of ways in the same data input. CRNN model structure comprises convolutional layer for feature extraction and LSTM layers for interpreting temporal structure and finally connected layers for output. These two methods include the measure of accuracy, precision, recall, and F1-score, as well as ROC curves. The last model is deployed in practice to execute all the assessments in an efficient manner, at a large scale while being highly available to all the stakeholders.

3.2. BLOCK DIAGRAM

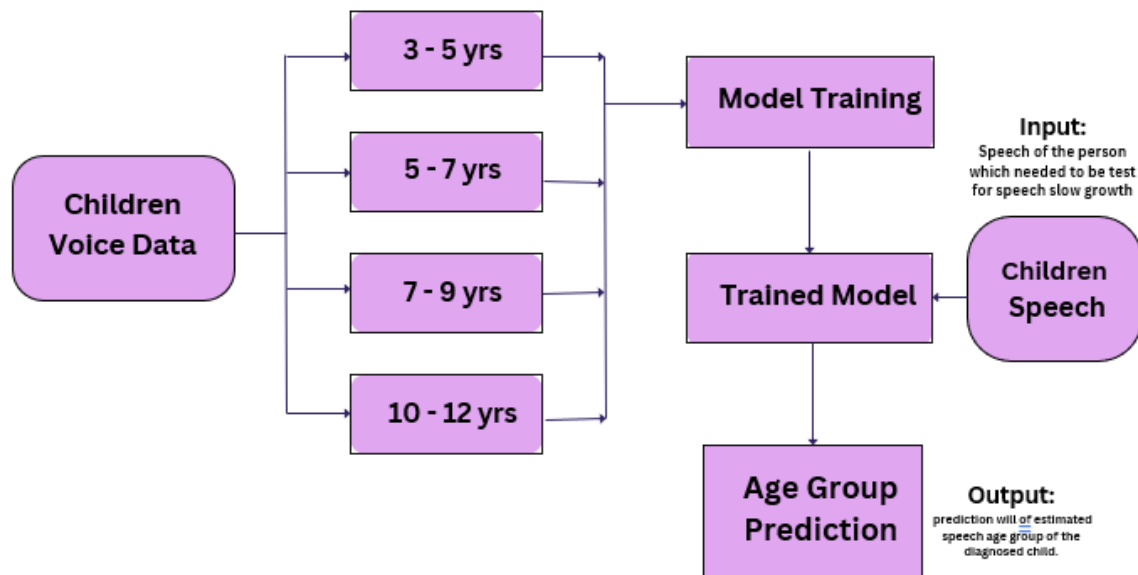


Figure 11 Block Diagram

3.3. DATA COLLECTION

In the data collection phase, details of how the dataset required for training and testing of the speech development assessment model was obtained and preprocessed are outlined. The dataset includes speech recordings from children aged between 3 and 12 years old and segmented by developmental age. All the used audio samples are also marked by the stage of speech development to introduce patterns within different age groups of users.

Feature Extraction: The basic features were extracted into the audio signals using mel-frequency cepstral coefficients (MFCCs).

Normalization: The features extracted were scaled for optimization so that the model is not sensitive to changes in speech quality of samples achieved.

Data Augmentation: Other methods such as time stretching, pitch scaling and adding noise were used to augment the data and thus enhance the model's performance in terms of data-dictated generality.

This strong preparation of the data guarantees that the CRNN model can successfully detect the kind of speech features desirable for proper developmental evaluation

3.3.1. *Speech Dataset Description*

The data available for use in this project consist of speech samples obtained from children, aged between 3 and 12. These recordings are then employed for the training and testing of the developed Convolutional Recurrent Neural Network (CRNN) used in the classification of the developmental phases of speech and identification of the possibility of speech disorders. The data set is explicitly tagged to the age groups to which they belong and hence befitting to the model for evaluating age specific differences.

Data Collection:

The database contains sound collected manually from all the children from local school. The dataset contains an audio of Urdu speaking children. All recordings contain only natural speech as the subjects were not given any cue to start speaking in front of the microphone. It's extensive in aspect to the features of development; it also takes into account the tone, pitch, and the rhythm as seen from the table below during the developmental stages.

Audio Format:

Recordings are produced in WAV type of file format with sampling rate of 16 KHz. High resolution format also preserves essential features of the acoustic signals namely, frequencies and amplitude variations of the speech signals for analysis. Overall, each recording ranges from 5-30 seconds, which maximized the data gathered for each age group.

Age Group Classification:

The dataset is divided into five age groups based on children's speech development milestones:

- **Group A:** Ages 3-5
- **Group 2:** Ages 5-7
- **Group 3:** Ages 7-9
- **Group 4:** Ages 10-12

The selected age groups have more than enough samples to establish the spoken language patterns associated with a particular developmental phase. These individual subclasses enable the model to distinguish age-related speech problems.

Feature Extraction:

In feature extraction procedure, the raw audio data is extracted into Mel-Frequency Cepstral Coefficients (MFCCs). MFCCs are popular in speech processing because they represent the original speech signal frequency accurately thereby making them suitable in modeling human speech. The extraction process involves the following steps:

- **Framing and Windowing:** The audio signal is divided into overlapping frames, each of which is windowed to reduce spectral leakage.
- **FFT and Mel Scale Filter Banks:** The Fast Fourier Transform (FFT) is applied to each frame, followed by a Mel-scale filter to emphasize frequencies relevant to human perception.
- **Log and DCT:** The logarithm of the filter bank energies is taken, and the Discrete Cosine Transform (DCT) is applied to generate the MFCCs.

The resulting MFCCs form the input to the CNN layers in the CRNN model.

Dataset Composition:

The current dataset contains about 350 samples subdivided into the four described age categories, with an overall audio length of about twelve hours. The dataset is structured as follows:

Dataset Split:

The dataset is split into three subsets for training, validation, and testing:

- **Training Set (60%):** Contains 60% of the total dataset, used to train the model and learn the speech patterns across different age groups.
- **Validation Set (20%):** Used for fine-tuning the model during training. This set helps to optimize hyper parameters and avoid overfitting.
- **Test Set (20%):** This set is reserved for evaluating the final model's accuracy and generalization ability on unseen data.

3.3.2. Data Pre-processing

The data set used for this project is made up of speech recordings of children between the age of 3 and 12 years. These recordings are employed for the training and testing of the Convolutional Recurrent Neural Network (CRNN) that is used to identify the speech development stages, and to identify signs of speech disorders in individuals. The data set has been accurately labelled with the respective age groups and hence the model can distinguish characteristics belonging to each of these developmental stages.

3.3.2.1. Mel-Frequency Cepstral Coefficients (MFCC) Extraction

MFCCs are again used in speech processing frequently as they imitate the human auditory model and give importance to the most crucial frequencies. The extraction of MFCCs from raw audio signals involves several steps:

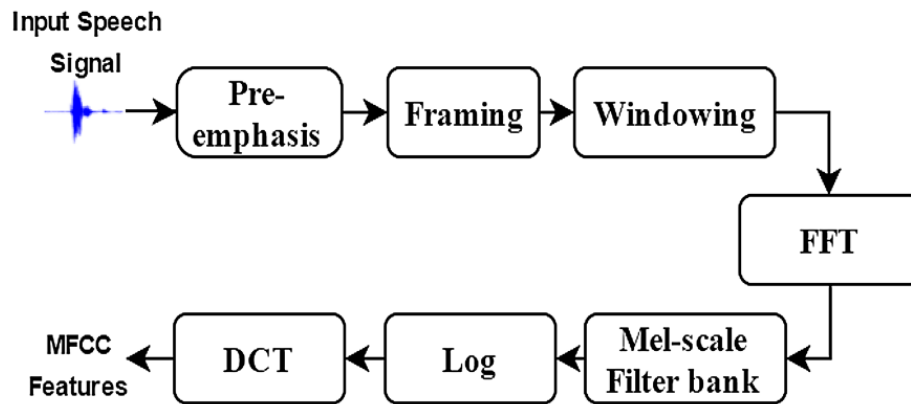


Figure 12 Block Diagram of MFCC

Framing and Windowing:

The raw audio signal is segmented into smaller overlapping frames of about 25ms because very short-term characteristics of speech signal are required for feature extraction. Each frame is then multiplied by a window function (typically the Hamming window) to reduce an effect termed as the spectral leakage in the next process.

Fast Fourier Transform (FFT):

Use of FFT is made to transform each frame from time domain into the frequency domain after windowing has being done. This step gives a spectrum which gives the frequency content for each frame.

Mel Filter Bank:

The FFT spectrum is then sent through a set of Mel filter banks which enhance some frequencies relative to the Mel scale. The Mel scale is a logarithmic measure that requires the bandwidth of a certain frequency to be proportional to its function's frequency. This step converts the spectrum into the Mel frequency domain helpful for speech processing and extracts only relevant features.

Logarithm of Filter Bank Energies:

To change amplitude scale of the signal to decibel scale for easy processing by the neural network a logarithm of the Mel-filtered energies is done.

Discrete Cosine Transform (DCT):

Finally, Discrete Cosine Transform (DCT) is used on the log filter bank energies to come up with the MFCCs, which can be used to represent the speech signal in a compact method. Generally, the

first 12 or 13 MFCC coefficients are used as features for this model. These processes made it possible to inevitably enrich hence improve the quality of dataset for the next model development.

3.3.2.2. Normalization

After extracting the MFCC features, the next step is normalization.

Normalization of the MFCC features that has been extracted. Standardization matters for directly resulting in that the features are scaled to an equal range to avoid in two cases: noise from recording conditions or recording environment affecting the model.

- **Mean Normalization:**

The MFCCs are normalized using equation 3 where the mean and standard deviation values is applied to each coefficient. This ensures that all the features in MFCC have a zero mean and unit variance hence easy for the model to capture.

- **Feature Scaling:**

These latter features are also unitized and normalized, preferably to a certain range such as 0-1 or -1 to 0. This step accelerates the speed of convergence by averting the indication problem and enhancing the model's performance.

3.4. MODEL ARCHITECTURE

We proposed a Convolutional Recurrent Neural Network (CRNN) to blend use of CNN for spatial representations with that of RNN suited for temporal representations. This architecture is particularly suitable in tasks that involve sequential data which for instance is the case in speech samples by age groups.

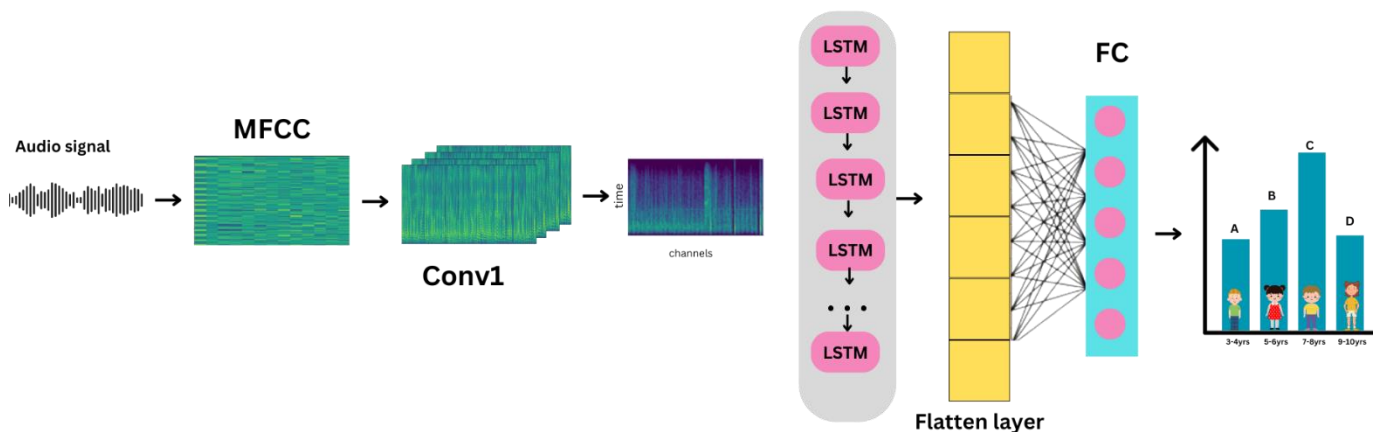


Figure 13 Proposed CRNN model

1. Input Layer (MFCC Features)

Another important part of the model input is Mel-Frequency Cepstral Coefficients (MFCCs), which is a popular feature in speech communication as it is short-term power spectrum of sound. The MFCCs reduce the speech signal into a feature matrix that defines the most important acoustical attributes of the sound spectrum important to the human ear.

2. Convolutional Neural Network (CNN) Layers: The first layer in the architectural progression is the Conv1 which forms the first stage in the architecture. This layer is tasked with identifying spatial patterns with the MFCCs where ever they may be present. The CNN layer convolves filter over the data and it extracts relevant local features from the MFCC matrix.

- Conv1 (1D Convolution): The first layer serves for applying filters to input MFCCs in order to produce feature maps. This assists in identifying the syllables of the speech signal with great facility.

Next after convolutional layer, the Max Pooling layer is used in order to reduce the dimensions of the extracted feature maps thus overly important features are preserved while the problem of overfitting is avoided.

3. Recurrent Neural Network (RNN) Layers (LSTM): On the convolutional layer, the data is forwarded to the Long Short-Term Memory (LSTM) layer. We also use LSTM layers which is type of Recurrent Neural Network (RNN) that good at catching long term dependencies in the sequences, such as in speech signals. They have memory, which is very important when approaching the task of analyzing how patterns of speech change over time.

- The LSTM layer takes in these features which are extracted from the convolutional layers and learns how these features evolve in the different time steps.
- The architecture comprises one LSTM layer because this layer is very useful in capturing the temporal nature of the inputs in the form of speech data.

4. Flatten Layer: Moreover, the output derived from the LSTM network are flattened to convert the high dimensional tensor to a vector for suitable classification. Down sampling is particularly critical for associating the output of LSTM with the fully connected layer as a classifier.

5. Fully Connected (FC) Layer and Output: Usually the final layer is the fully connected layer (FC) which take flatten the output and give out the final prediction. This layer gathers the learned features from the LSTM and delivers the last categorization for the age groups.

- **Output Layer:** We take the number of units in the output layer as 5 with each of them corresponding to a specific speech age group: 3-5 years, 5-7 years, 7-9 years and 10-12 years. The last layer that applies the soft max function makes a contribution to the fact that through the discriminant function the model gives a probability distribution for these four classes of age. The final output is one of the five developmental stages (age groups) according to the features of the speech.

6. Output Groups

The model classifies speech samples into the following 4 age groups:

- **Group A:** Ages 3-5
- **Group 2:** Ages 5-7
- **Group 3:** Ages 7-9
- **Group 4:** Ages 10-12

This architecture can bring efficiency for the speech classification processes since it learns the spatial features from the convolutional layers and temporal sequence learning from the LSTM layers. The model is highly accurate for predicting the developmental age group from spoken speech input.

3.4.1. CRNN Overview (CNN + GRU/LSTM)

While selecting the architecture for this project, the Convolutional Recurrent Neural Network (CRNN) was adopted because it can capture the spatial and temporal features of the of speech data. Since speech signals comprise both time-varying spectral features and time-domain features, the

CRNN is specially constructed to address the input signals' temporal and spectral characteristics with the incorporation of CNNs and RNNs.

1. CNN for Spatial Feature Extraction

In this project the speech data is converted to MFCC features because MFCC is a more compact representation of speech in time-frequency domain. CNN layers are particularly effective in capturing the local spatial patterns in these MFCCs, such as:

- **Pitch contours:** Example of the model of how pitch changes in time.
- **Formants:** Pitch of the signal most dominant frequency or pitch and its intensity relative to other frequencies.
- **Temporal shifts:** Small changes in the way that features reflect their patterns of change.

By the matrix FFT's convolutional layers the model can identify these crucial acoustic cues that relate to the speaker's developmental stage from the MFCC feature matrix. Three convolutional layers are chosen to enable the model to learn simple features at the first layer and then to combine them in a more complex manner at deeper layers.

2. RNN (GRU/LSTM) for Temporal Dependency Modeling

Speech is a sequential data source, it can be seen that the straightforward CNN layers, although they are effective in capturing spatial patterns, cannot model temporal inter-frame dependencies in the sequence. For instance, gradual change of speech characteristics (for example, the gradually increasing pitch or prolongation of vowels) may indicate certain age aspect.

To capture these dependencies Recurrent Neural Networks (RNNs) are used after the CNN layers. In particular, there should be GRU (or, in fact, LSTM) units that could address the sequence of features provided by the CNN layers. GRUs are elected due to their applicability in training, as well as due to the exclusion of the issue of vanishing gradients, inherent to standard RNNs.

- It needs to maintain some past information through the whole sequence of speech data
- Exclude pre-class noise as it is essential when considering separating different stages of speech development. The ability to model the temporal dependencies between frames in the sequence.

Using these recurrent layers the model is able to analyze the dynamics of features of speech, which is crucial for identification of patterns related to different age groups

3. Integration and Classification

The CNN-RNN (LSTM) integration empowers the CRNN formulation for carrying out classification by using speech as a basis. The CNN layers are mainly used for identifying short-term features within the audio units and the LSTM layers are used for recognizing long-term dependencies of the signal during its given time frame.

Last, the fully connected layer of the model uses a softmax function for the outcome of the probability of the uttered speech sample belonging to one of the initially defined five age groups. One major advantage of this form of integration is that the CNNs and the RNNs work together to capture both short-term and long-term features of a speech which results in a sound classification output.

CHAPTER IV

SYSTEM DESIGN

In this chapter, we will explain how to deploy the web application that has been developed for Sonic Pinnacle that includes a front end created with the React framework, back end created with the Flask framework, and a database created with Firebase. The front end captures children's voices and store their audio files in the back end where the Deep Neural Network (DNN) model enhances for developmental stage prediction by evaluating child's speech patterns. It is then compared to an actual class in order to determine the efficiency of the model. This chapter will provide end to end workflow starting from front end, back end, database integration and deployment.

4.1 OVERVIEW

Sonic Pinnacle web application is intended to recognize and forecast developmental stages of children's speech using M-Level models. It assists parents, educators and speech specialists evaluate the developmental process of a child's speech through promoting a user-friendly environment for recording and analyzing clips of the speech impairment. The purpose of the application is the age estimation of children by means of acoustic features derived from children's speech with the help of MFCC and DNN.

The application consists of two major components: a front end created in React which is basically the user interface where the user uploads an audio file or records a new one and a back end built using Flask, TensorFlow where the back end produces predictions based on the child's speech development.

Key Features:

1. **Audio Recording and Upload Interface:** It has a friendly user interface where a parent or a guardian can either record samples of a child's speech within the application or upload new audio samples. This is because most popular formats, including MP3s and WAVs are supported by the application. The layout of the webpage is one way of making it very simple for users and this is done through the provision of very simple instructions on how to record and upload the audio files.

2. **Speech Analysis for Developmental Stage Prediction:** The basis of the application is in the performance of the recorded or uploaded audio data through a machine learning algorithm that is designed to analyse speech patterns. This prediction is done using MFCC feature extraction after which classification is done using a Convolutional Recurrent Neural Network (CRNN). The system analyses the child's speech and maps it into one of the developmentally defined phases out of the phases which are derived from the stored audio data.
3. **Real-Time Feedback:** When the audio file has been passed through the back end of the system, the user gets feedback almost instantly. The feedback that is provided by the program includes the predicted age of the child based on the child's manner of speaking; the sensitivity level of the model's prediction. The outputs are therefore presented in figures which makes it easy for parents and caregivers to understand the prediction made.
4. **Data Storage and Progress Tracking:** Every user's information is stored on Firebase to maintain recording of the child's speech in a progressive manner. In this case, parents and specialists can monitor the prior work submitted and the outcome or lack of it in an endeavor to notice enhancements or speech difficulties impelling for remedial steps.
5. **User Authentication and Security:** For authentication, the application uses Firebase, and to check the users' access to the information about children's speech data, the application requires undergoing the identification process. This makes certain that any interaction between the graphical user interface, middleware, and database is completely secure.

4.2 FRONT-END:REACT

The front end of our web application, Sonic Pinnacle, is developed using React, a widely used JavaScript library for building dynamic and responsive user interfaces. React's component-based architecture allows us to modularize the different sections of the web application, making the interface scalable and maintainable. The main purpose of the front end is to allow users to capture speech data, provide tags or descriptions for the speech data and make use of real-time prediction based on the child's speech development stage. This section will explain the main components that

have been developed on the front end of the system, with regards to the user experience/interface design.

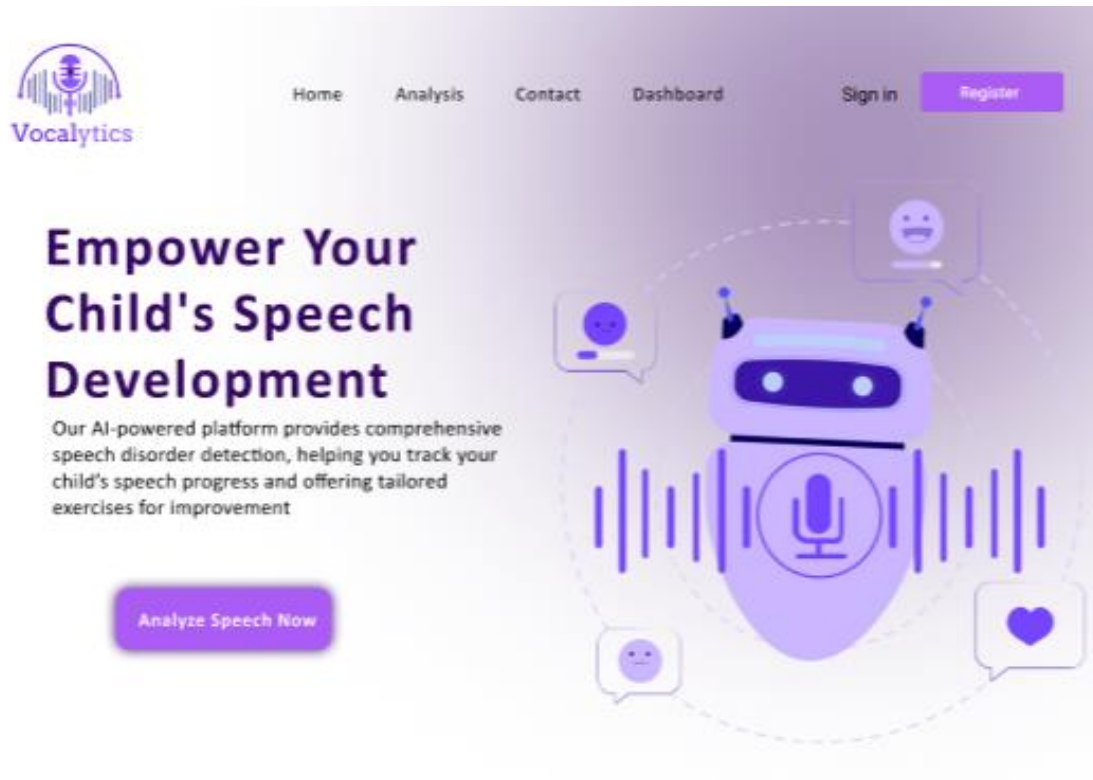


Figure 14 React Frontend Application

4.2.1 Features of the React Front End

Audio Recording Interface:

One of the core features of the React front end is the audio recording interface. The users (often parents or clinicians) can feed a child's speech directly within the context of the web application. The Web Audio API is used to record the audio and then we work on it in real

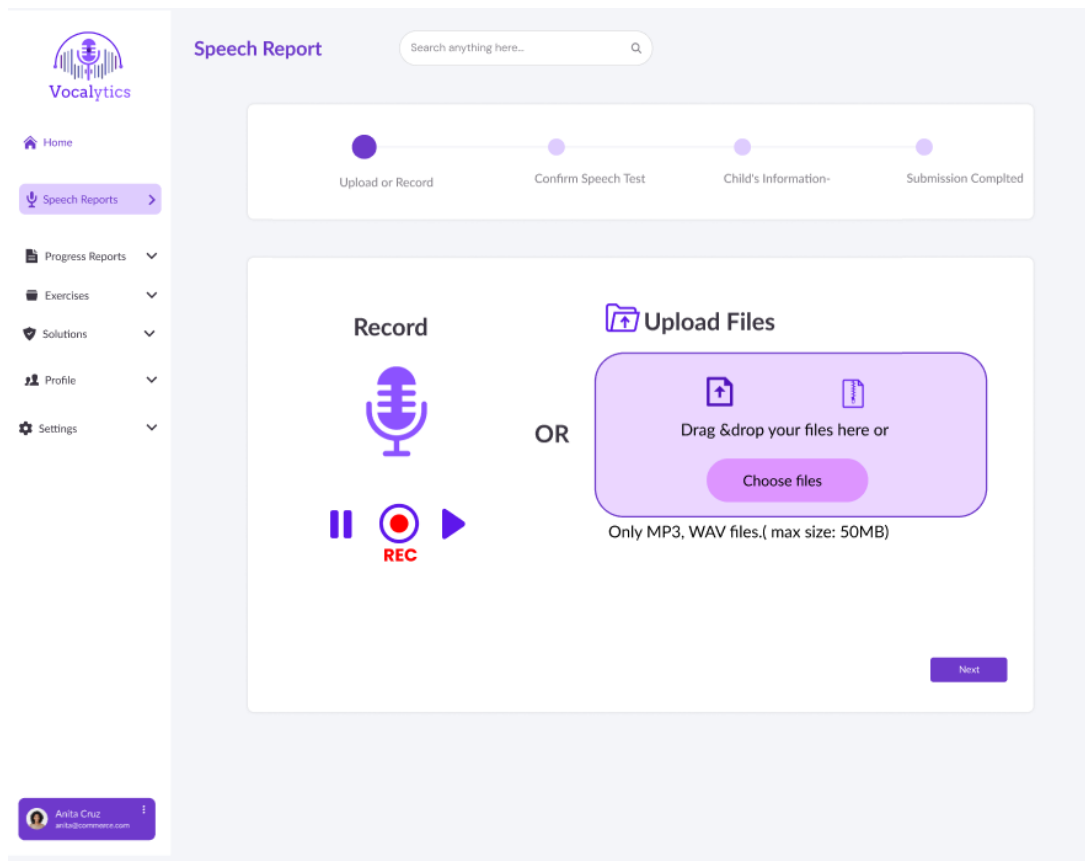


Figure 15 React Speech Report

time. The recording is activated by an ordinary button that records the speech, then the speech is digitized, encoded in.wav and ready for transferring to the server. After the recording is complete, the audio file is save in the Local Storage of a browser, ready for additional interaction or immediate and direct submission to the back end for processing.

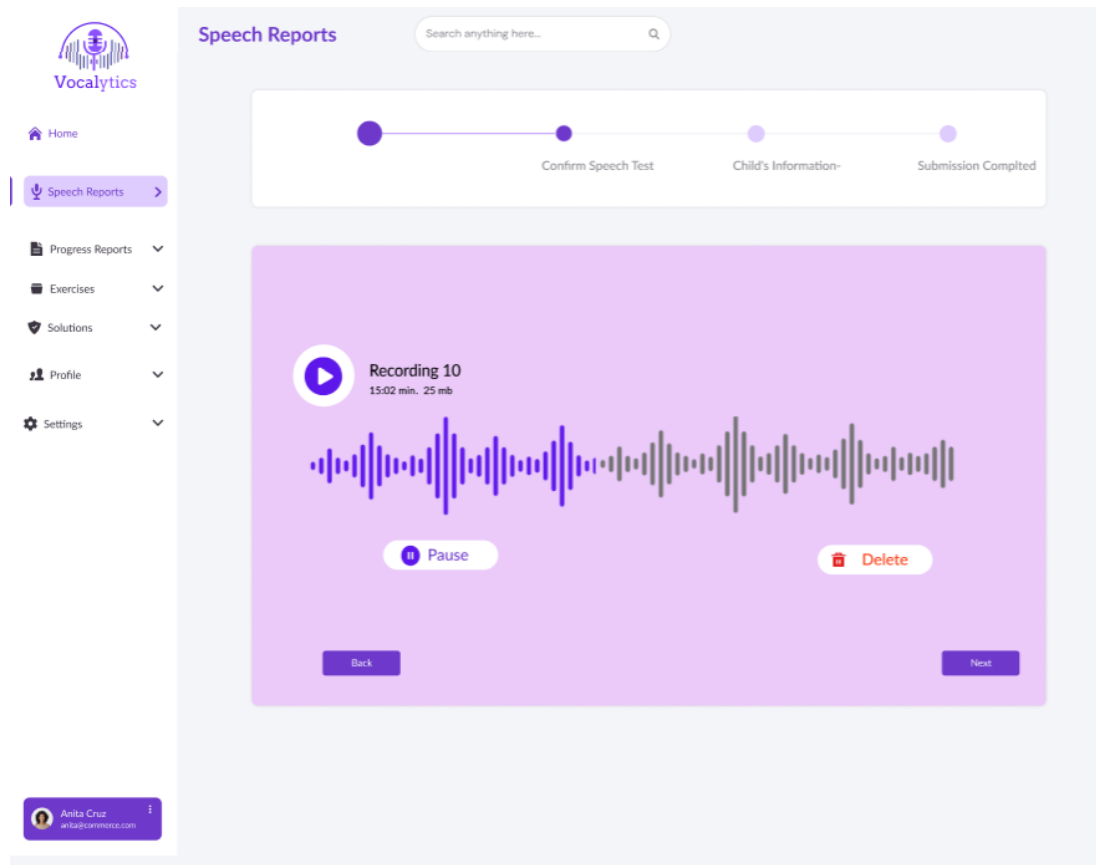


Figure 16 Audio Playing

The interface is designed to be intuitive, with visual feedback provided during the recording process (such as showing the waveform or an animated recording icon), ensuring a user-friendly experience. This visual representation ensures that users know when the recording is active and when it has stopped.

Form Handling:

Besides speech recording, users have to provide additional information relevant to the analysis in form of metadata input. This may include the name of the child, the child's age, and even the date on which the speech test was done. The form is handled using the tracking mechanisms provided for in React State Management to track the progress of the input as it happens. This metadata is important because after training, the determined developmental stage will be compared with the child's age to assess the accuracy of models. The form also has various validation mechanisms that will not allow the user to submit the form without filling some fields such as age and name. This information is

added in the metadata with the actual audio file via an HTTP POST formatted by structuring.

The screenshot displays the 'Speech Reports' section of the Vocalytics application. On the left, a sidebar contains navigation options: Home, Speech Reports (highlighted), Progress Reports, Exercises, Solutions, Profile, and Settings. The main area features a 'Speech Reports' header with a search bar and a progress bar indicating the current step: 'Upload or Record'. Below this, the 'About me' section is titled 'Mandatory information' and contains four input fields: 'Child's name' (text input), 'Child's Age' (dropdown menu), 'Child's Date of Birth' (date picker), and 'Test Date' (text input). A 'Next' button is positioned at the bottom right of the form.

Figure 17 User Form

Real-Time Feedback:

Besides, there are other fields which need to be filled for the speech analysis: Navigation is an important element for the continuous metadata input concerning the speech recording. These include the name of the child, age of the child and date of when the child was given the speech test. Regarding form handling, basic functionalities of React state management are used to track the input as it happens. This metadata is important because the actual developmental stage which the child is in will be the benchmark against which performance of the predicted developmental stage will be measured.

The form also, for example, has features to validate that some of the fields including age and name are completed before the form is submitted. That is why when sharing its content,

the metadata is sent alongside the audio file using an HTTP POST request with a structured format.

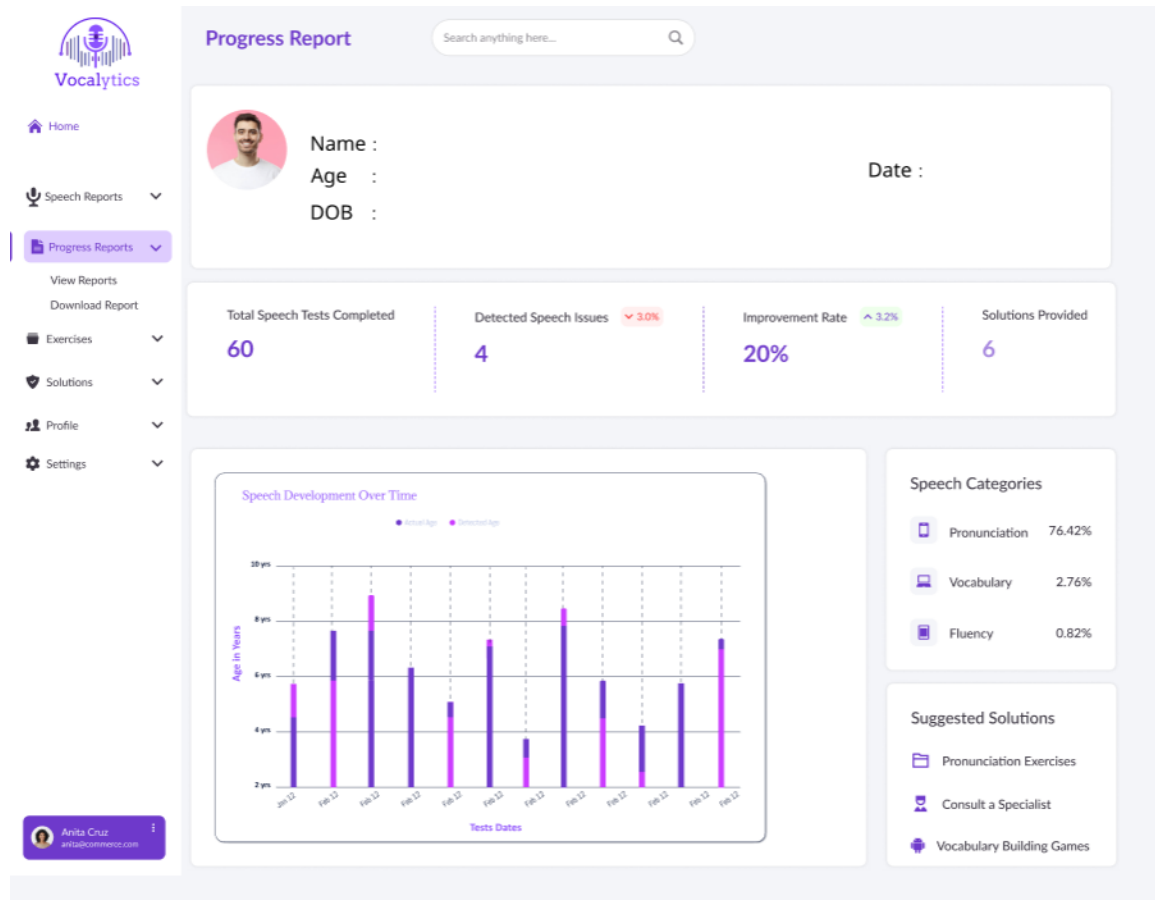


Figure 18 Dashboard

The feedback is real-time, which makes the system as interactive and engaging as it can be for the particular user who might require a quick sign of the child's speech pattern. The feedback section also gives recommendations based on the outcome, which can be a course of action that users should take in case of detected speech developmental problems or go to a specialist.

4.2.2 React to Flask Communication

The interaction between the React front end and the Flask back end is done through the use of RESTful APIs. The front end is designed to send an HTTP POST to the back end as soon as the

user submits the speech recording and metadata; this POST contains the recorded audio in WAV or MP3 format with metadata input by the user (age, name, etc.). The submission process guarantees the safe transfer of the data as form data, and adheres to data integrity. The prediction is then sent back as a JSON response once the back end processes the audio. The front end then takes this response and breaks it down into the correct predictions, confidence levels and recommendations according to the model.

```

plt.figure(figsize=(10, 4))
1/1 ██████████ 0s 432ms/step
5
INFO:werkzeug:127.0.0.1 - - [24/Oct/2024 23:17:12] "POST /predict HTTP/1.1" 200 -
1/1 ██████████ 0s 60ms/step
7
INFO:werkzeug:127.0.0.1 - - [24/Oct/2024 23:18:23] "POST /predict HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [24/Oct/2024 23:21:56] "POST /predict HTTP/1.1" 400 -
INFO:werkzeug:127.0.0.1 - - [24/Oct/2024 23:21:58] "POST /predict HTTP/1.1" 400 -

```

Figure 19 Backend Request

4.3 BACK-END: FLASK

Sonic Pinnacle speech analysis application is developed on Flask a fast, lightweight and flexible web framework of Python for the back end of the application. The back-end system is to receive the file upload, process the speech data and return the age group and the corresponding confidence score. The application combines Flask with Tensor Flow for its speech age classification using deep learning approaches.

4.3.1 Flask API Routes

The Flask API is going to perform multiple operations, while the /predict route is the core point of the application, which processes the speech recording. It is available for HTTP POST with uploaded audio files and metadata (actual age of the child, for example) to return predictions and comments.

The /predict route handles several operations:

- **Receiving audio files:** It checks for the presence of an uploaded audio file in .wav or .mp3 format.

- **Saving and processing the file:** The file is temporarily stored and passed through the audio processing pipeline, where it is converted into a spectrogram.
- **Running predictions:** The processed audio is fed into the deep neural network model, which generates predictions about the child's developmental speech stage.
- **Returning results:** The predicted age category and confidence score are returned as a JSON response.

To perform this the audio file is converted to a spectrogram using the librosa library and then the spectrogram is given to the DNN model to predict the developmental stage of the child from the speech they make. The back end then compares the model's predicted class (age category) with the actual age the user inputted.

The Flask API route that handles this process is structured as follows:

```
@app.route('/predict', methods=['POST'])
def predict():
    if 'file' not in request.files:
        return jsonify({"error": "No file part in the request"}), 400

    file = request.files['file']
    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        audio_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(audio_path)

        # Convert audio to spectrogram
        spectrogram_path = create_spectrogram(audio_path)
        # Predict the child's age group
        img = tf.keras.utils.load_img(spectrogram_path, target_size=(200, 200))
        img_array = tf.keras.utils.img_to_array(img)
        img_array = np.expand_dims(img_array, 0)
        predictions = model.predict(img_array)

        # Extract the predicted class and confidence score
        predicted_class = class_names[np.argmax(predictions[0])]
        confidence = np.max(predictions) * 100

        # Clean up the uploaded files
        os.remove(audio_path)
        os.remove(spectrogram_path)

    return jsonify({
        "predicted_age": class_to_number[predicted_class],
        "confidence": confidence
    }), 200
```

Figure 20 API

The code uses TensorFlow to load and run the DNN model, which predicts the child's developmental stage based on the audio file. The spectrogram generated from the audio is resized and converted into an image format suitable for input into the neural network.

4.3.2 Deep Neural Network Model

The deep neural network (DNN) adopted in the project is intended to estimate age group of a child from speech signals. Being based on spectrograms, the model learns precise time-frequency characteristics of the speech signal, which is necessary for the intended solution.

Input: The input to the DNN is a spectrogram image with dimensions of 200x200 pixels. This ensures that the audio features are represented in a format suitable for image-based deep learning models.

Output: The model provides probabilities of its input falling into the pre-defined age ranges class. The age with the maximum probabilistic value is chosen as the predicted age and also the associated probability score.

Model Input and Output:

- **Input shape:** The spectrogram image has the size of 200x200 and value range of (200, 200, 1).
- **Output:** A softmax layer provides the expected age, as well as the reliability of this prediction.

The spectrogram image is important in age classification since it contains both time and frequency characteristics of the speech of the child. Using this information, the model is able to determine the correct developmental stage of the child.

```

# Model Loading and prediction
model = tf.keras.models.load_model('speech_age_model.h5')

def predict_speech_age(spectrogram_path):
    img = tf.keras.utils.load_img(spectrogram_path, target_size=(200, 200))
    img_array = tf.keras.utils.img_to_array(img)
    img_array = np.expand_dims(img_array, 0) # Expand dimensions for batch input

    predictions = model.predict(img_array)
    predicted_class = class_names[np.argmax(predictions[0])]
    confidence = np.max(predictions) * 100
    return predicted_class, confidence

```

Figure 21 Model Interference

4.4 FIREBASE INTEGRATION

The Sonic Pinnacle contains the speech analysis application, and the main cloud-based backend for user authentication and data storage is Firebase. Firebase provides a clean integration into web and mobile environments which makes it ideal for managing user storage, authentication and storing the results of the predictions. Also, Firebase offers synchronized capability and users can view changes in their data without the need to refresh. This integration helps to simplify the administration of User accounts and secure storage of the predictions and metadata successfully.

4.4.1 Firebase Authentication

One of the key aspects of Firebase integration is managing **user authentication**. Firebase Authentication enables the application to ensure that only authorized users (parents, guardians, or authorized professionals) have access to sensitive data such as the child's speech records, prediction results, and personal information. Firebase provides multiple methods for authentication, including email and password authentication, social logins (such as Google or Facebook), and anonymous login.

User Account Management:

- **User Registration:** When a new user signs up, Firebase Authentication manages the process of securely storing the user's credentials (email and password) and creating a unique user ID in the Firebase system.
- **Login and Logout:** Firebase predetermines the management of login and logout processes and provides tokens for the authenticated users and their sessions do not expire if they did not log out or if session expired. They are used to sign each request to the Firebase backend valid.
- **Authorization:** Firebase allows control options for users where an admin can provide more features to the user based on the role of the user such as admin, normal user, or a guest. This can allow only specific staff to access specific options or info, and eliminate the privacy of the kid's speech data.

4.4.2 Storing Results and Metadata

Once the analysis on the speech is complete, the application saves the obtained prediction result coupled with certain user metadata in Firebase's Cloud Fire store. This also ensures that all the results of the predictions, together with other essential user details are safe and retrievable in case of additional analysis or in situations where a report needs to be generated.

The stored data includes:

- **Predicted Age Category:** The age at which the child is hypothesized to speak based on the identified referring expression and using the DNN model.
- **Confidence Score:** The level of confidence expressed by the model, so the user is aware of how accurate the prediction being made is.
- **Actual Age:** The real age of the child as the input that the user enters in order to compare the predicted and actual developmental levels.
- **Speech Metadata:** This is all in regard to the recording date, the length of the audio sample and every other factor that may be useful in future studies.

Firebase also offers an ability to work on real time basis so once a prediction is made, it is updated immediately on the user's panel. This makes it possible for parents, teachers, or clinicians to have a record of how a child uses their voice over some time, with previous data collected being retrievable for comparison.

4.4.3 Data Tracking and Analysis

One of the main advantage of the Firebase is that it allows you to analyze and put the stored data in the form of a visual. By tracking predictions over time, the app can provide useful insights such as:

- **Trends in speech development:** From the predicted ages and actual ages, parents and Child-care professionals can determine the progress of the Speech & Language Development.
- **Identifying patterns or issues:** Also, the system can signal that a child may require help in some particular areas, for instance, speech therapy , if the results of the program showed low confidence levels or detected considerable differences between the expected and actual ages.
- **Reporting and future assessments:** All data collected is stored in Firebase where it can be exported and used for further clinical evaluations or research contributing positively to tailor made speech development.

4.5 DEPLOYMENT

The application for Sonic Pinnacle is hosted and operated locally in a localhost set up at the moment. Though the application has not been hosted in a cloud platform like Heroku, the app works fully on the local environment which will be used to run and demonstrate the application. In the following part, we will explain the measures that have been taken down below in order to facilitate local deployment and setup.

4.5.1 Gradio Deployment

Gradio is an open-source Python framework that makes it easy to build quick interfaces like chatbots, voice-activated bots, and even full-fledged web applications to share your machine learning model, API or data science workflow with clients or collaborators.

We have deployed our deep learning model on Gradio as it is comparatively easy to deploy deep learning models on, it provides an easy interface and easy deployment on cloud so we can easily access the web app from the internet. It deploys the web app on HuggingFace which provides cloud compute for deploying web apps. Below are the figures for the web app and chatbot which are prompted to give responses related to speech-related problems.

SONIC PINNACLE
Advancing Age-Linked Speech Analysis through AI

Sonic Pinnacle is an innovative AI-powered tool designed to assess and analyze children's speech development. By leveraging advanced machine learning algorithms, we provide insights into speech patterns, helping parents and professionals understand a child's linguistic progression. Our cutting-edge technology transforms voice recordings into detailed spectral analyses, offering precise age-group predictions and developmental insights.

Age Prediction | Speech Development Support

Upload Child's Voice Recording

Drop Audio Here
 - or -
 Click to Upload

Predicted Age Category

Prediction Confidence

Child's Name

Child's Age

Predict Age Category

Prediction History

Name	Real Age	Real Age Group	Predicted Age Group	Prediction Confidence

Figure 22 Gradio WebApp for Model Deployment

In figure 23 for the chatbot instructed to give responses related to child-related speech therapy and diseases and also help understand the model results.

SONIC PINNACLE
Advancing Age-Linked Speech Analysis through AI

Sonic Pinnacle is an innovative AI-powered tool designed to assess and analyze children's speech development. By leveraging advanced machine learning algorithms, we provide insights into speech patterns, helping parents and professionals understand a child's linguistic progression. Our cutting-edge technology transforms voice recordings into detailed spectral analyses, offering precise age-group predictions and developmental insights.

Age Prediction | Speech Development Support

Speech Development Assistant

Hello! I'm here to assist you with understanding your child's speech development. Please provide details about your child's voice recording or ask any questions about speech milestones.

hello

Hello! It's great to connect. To help me understand your child's speech development, I need some information. Please tell me your child's name (or initials for privacy), age, and when you'd like to share the voice recording. Knowing this will help me interpret the AI's age prediction from the voice analysis and give you the most relevant advice. We'll then discuss any concerns and explore strategies to support your child's speech journey.

Your Question

Send

Clear

Figure 23 Gradio Chatbot

4.5.2 Local Deployment

Local deployment is a critical step in the development of the Sonic Pinnacle web application. In this stage the front end which is based on react, the back end utilizing Flask and the database from Firebase can all work together in a local environment for testing, bug fixing and implementing other features before making the application live.

React Front End: The front end that is developed on React is served locally by running the React development server on the developer's machine. The process starts by using the command `npm start`, which serves the React app on a local host, typically accessible at `http://localhost:3000`. The speech recording, along with inputs for additional metadata such as where the recording is coming from (the child's name and age), and all submission interactions to the back-end API are handled through the React interface. The local deployment helps developers to instantly review the application's UI and make changes in the same testing environment.

Flask Back End: The Flask server only listens to HTTP POST messages from the React front end, processes the corresponding audio files, and sends the predictions. The local deployment of flask is done by using the command `python app.py`. On the flask side the functions are written that act as routes through which they handle the requests from the front end and the API. It also holds the audio files temporarily for processing and manages the interface to the TensorFlow model for the generation of predictions on the age of speaker.

Firebase Integration: Although the application is deployed locally, Firebase is still used for authentication and data storage. . Firebase also makes it possible to store user data and the metadata including the predicted results in a local application environment. For the local initialization, the Firebase connection contains the path of data flow for developers, debug the processes of the database, and test authentication processes. It is set up to communicate with Firebase's real-time database, so we can achieve end to end testing before we even put it on a remote server.

Steps for Local Deployment:

1. **Flask Setup:** The Flask back end runs locally on `http://localhost:5000`. As with most Python tools, Flask is initialized by being run out of a command line; in this case, the command is ``python app.py`` when the Flask app has been fully configured. Flask manages incoming

requests, on the frontend which is built in React, and then processes the uploaded audio files and returns the predictions.

2. **Firestore Integration:** In the local setup as well the Firestore is used for keeping the databases of the users as well as the result. Firestore integration guarantees that all predictions and the associated metadata are kept safe as well as configurable for real-time effectiveness on the results.
3. **Testing and Local Usage:** The whole application program resides in the developer's PC and other users cannot access it, this makes developers easily tweak different features of the application as they try their application locally without having to upload it to a remote server. It is important to note that both the front end and the back end are equally in harmony with a view of working within the local environment. This configuration enables the system to work as a production while at the same time being fully developed and tested locally.

4.5.3 Continuous Integration

As for process arrangements for the development, in addition to local deployment, CI is used by adhering to the best practices for the development process. CI helps in version control testing and even development teams can control the versions of the software easily.

Version Control: For development, version control, Git is used where developers work collaboratively on the application's development processes. Any modification made to the front end (built with React) or back end (built with Flask) are committed to a Git repository. This makes sure that several developers can work on the documents, without the risk one overriding work done by the other. Any addition or modification is done in the live branch and checked out after getting committed and reviewed. Pull requests are employed for the exact reason that requires each new code change to undergo an extensive review prior to being incorporated into the project's code base.

Automated Testing: CI also incorporates, the testing of both front end and back end using automated scripts. Testing frameworks have to be used to ensure every new feature meets expectations, while other features do not present any issues. For instance, unit tests call specific components of the Flask API, for instance the /predict endpoint to determine how it processes different audio files and produces the right results. Likewise, the React front end is checked in the

way of usage to guarantee that all the UI element performs suitably when it communicates with the back end.

Deployment Stability: CI continues to check the status of the application and make certain that it is up to date on the system at all times. Every update is made in the local environment and only then merged to the repository with proper documentation. After all tests have passed, the changes developed are committed back to the repository, and the local environment deploying the applications is synchronized with the most current version.

CHAPTER V

EXPERIMENTATION AND RESULTS

5.1. MODEL TRAINING AND TESTING SET

The training and testing phases of the CRNN model are critical for ensuring the model's ability to accurately classify speech samples into the five age groups. This section provides an in-depth explanation of the setup used during the experimentation phase, detailing the configurations, environment, and evaluation methods used to measure the model's performance.

5.1.1. Software Configuration

Software Setup:

- **Python:** The entire pipeline was developed using Python, which facilitated the integration of various libraries and tools.
- **TensorFlow/Keras:** The deep learning framework **TensorFlow** with **Keras API** was used for building, training, and evaluating the model. This allowed for seamless implementation of the CRNN architecture.
- **VS Code:** For managing the codebase and tracking experiments, **Visual Studio Code** (VS Code) was used as the primary IDE. Version control was managed through **Git**, ensuring that different model configurations and code iterations could be easily tracked.

5.1.2. Dataset Split (Training, Validation, Testing)

Data, consisting of the MFCC features obtained from the speech signals of children in the age group of 3 to 12 years, was split into three sets to avoid cross-validation and an overfit model.

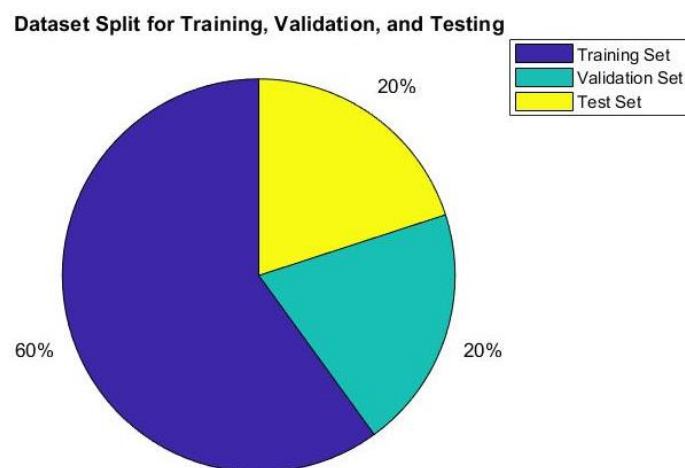


Figure 24 Dataset Split for Training , Validation , Testing

- **Training Set (60%):** Out of every 100 volumes, 60 volumes were used to train the model. The training data was utilized for back propagating the weights of the model as per requirement. During this phase of the model, it was able to understand patterns of relationship between the features in terms of MFCC and the age group.
- **Validation Set (20%):** In training the model, 20% of the data was set aside as a validation set to help evaluate the model's ability to generalize on unseen data and to fine-tune hyperparameters. The aim of using a validation set consists in avoiding overtraining on a training set by giving feedback on a model after each epoch
- **Test Set (20%):** Finally, the last 15% of the data set was used for the final testing. This test set was utilized solely after structure learning completed, which makes it possible to estimate how well the model will work with data that it has not encountered yet.

5.1.3. Preprocessing (MFCC Extraction, Normalization)

In order to improve the training of the model, it is required to 'clean' the data that is fed into the program. The preprocessing steps included MFCC extraction and normalization:

- **MFCC Feature Extraction:** The raw audio signals were analyzed through the Mel-Frequency Cepstral Coefficients (MFCCs) that characterize the frequency content of the speaker's signals. MFCCs are extensively employed in speech processing because they are capable of encapsulating the required information of speech signals efficiently.
- **Frame Length:** The audio samples were segmented into frames of 25 milliseconds with hop length of 10 milliseconds.
- **Number of Coefficients:** In total, 64 MFCC coefficients were taken from each frame, and as a result each audio sample was described by 2D matrix reflecting time-frequency characteristics of speech.
- **Normalization:** As for the next feature extraction, the obtained MFCC features were normalized to improve internal consistency of the dataset. This was done by normalizing the MFCC features to have mean of zero and standard deviation of one. Normalization is important because all values of features that are used when training models should be in

the same range, such that some features don't swamp the learning process because of their scale.

5.1.4. Hyperparameters Tuning (Batch Size, Learning Rate, Epochs)

The model of CRNN highly depended on the choice of hyperparameters. After several trials, the following hyperparameters were selected for optimal training:

- **Batch Size:** The number of samples per gradient update batch in all experiments was 64. Compatibility of batch size with these conditions was the rationale for selection of the chosen batch size. Greater values of were attempted but the convergence rate took longer and the accuracy lower than with the value of 0.001.
- **Learning Rate:** They began with a learning rate of 0.001. A learning rate scheduler was used that decreased the learning rate by 0.5 if validation loss did not subsequently obtain a lower value for a total of 5 times. Due to this, the model was capable of making relatively massive updates to the weights during the training phases while refining them with relatively small updates during the later phases of the training.
- **Epochs:** The model was trained up to 80 epochs but the training stopped during the epoch when the validation loss was at its smallest. This allowed for stopping of training process after the commencement of overfitting which was determined by the high validation loss.

5.1.5. Early Stopping and Regularization Techniques

To prevent overfitting and ensure that the model generalizes well, several regularization techniques were employed:

- **Early Stopping:** The use of early stopping was done using the validation loss. In fact, if validation loss did not decrease over the next ten epochs, the training process was stopped. This made sure that the model does not began to train beyond the optimal time it has took to find the best model in the model selection process.
- **Dropout Regularization:** In the fully connected structure, dropout was used to minimize the overfitting problem that occurred during training. For each training case a dropout ratio of 0.5 was applied , 50% of the neurons were randomly turned off, so that the model could rely only on the most important features.

- **Batch Normalization:** After every convolutional layer , a batch normalization technique was applied. This normalization technique aids to make and increase the training stability and speed by making the input layer to be of one distribution rather than distributing it by learning rates on each layer.
- **L2 Regularization:** L2 regularization was used to Apply to the weights since large weights are not encouraged and hence adding a cost function to control the weights and reduce them. This is good for the model because it encourages its weights to stay small, so that overfitting can be prevented.

5.2. EXPERIMENTAL RESULTS

5.2.1. Overall Model Accuracy (Train, Validation, Test)

In this section, we aim to assess the general performance of CRNN model on the training set, the validation set, and the test set. The accuracy metric is considered to be very important in giving details on how the model is capable of correctly classifying speech samples into given age categories. 60 percent of data were used for training, 20 percent for validation and the remaining 20 percent for testing to assess the performance during different stages of the model's development.

Training accuracy: During the training phase the model got familiar with pattern from training data set and the percentage of correct answers or Accuracy was 94%. The hypothetical validation set that was incorporated and utilized during training but was composed of distinct data was used to estimate generalization ability free from overfitting. The performance increased and reached validation accuracy which indicates that the model keeps the acceptable level of generalization. Last but not the least, on the completely unseen test data the model obtained test accuracy , so proving the ability of the model that it performed very well and does not need to undergo the training process all over again in case of any new data set. This closeness of 'training, validation and test accuracy' suggest that the CRNN model that has been developed is adequate for the classification of age from speech data that it has not been trained on.

5.2.2. Confusion Matrix Analysis

Confusion matrix is one of the most effective methods that are used to evaluate the performance of the CRNN model as it gives the number of correct as well as incorrect classification of the ages. It provides a detailed picture to the extent that the model is able to distinguish between various classes of speech samples. The columns of the matrix presented here represent the predicted age group labels while the rows represent the actual labels. The number of predictions in each cell of the matrix corresponds to a certain Actual and Predicted label.

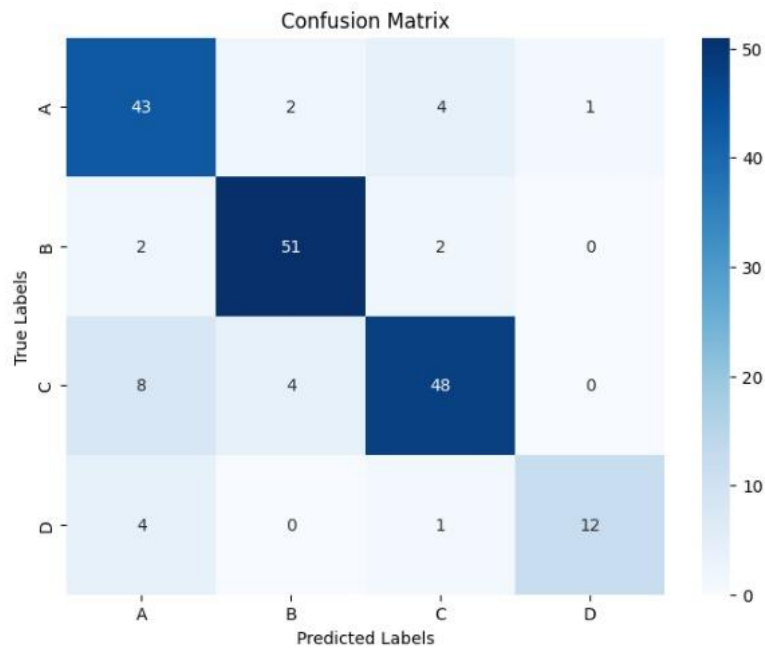


Figure 25 Confusion Matrix

From this matrix it is clear that the model we used yield good result and a large number of samples were correctly classified in each age group, most of the classification are on the diagonals which shows that most of the results are correct classification.

As we can see from the confusion matrix the model is performing very well having very few classes being misclassified. And every class is predicting most of the audio files correctly. In general, the confusion matrix represents the fact that the proposed CRNN model performs well dealing with age classification based on speech input data with a low number of misclassifications

and provides the ideas of further increasing specific accuracy by means of tuning or using more data augmentation.

5.2.3. Precision, Recall, and F1-Score Datasets

We analyze the precision, recall and F1-score on five groups of age (A to D) formed by the proposed CRNN model.

5.2.3.1. Precision

We have observed that precision is a crucial metric that measures the model's accuracy in predicting a specific class without including incorrect classifications. The best performance by the model is observed for Class B (5-7 years), for which all 51 cases are correctly predicted from 55. In the second place Class A (3-5 years), then the Class C (7-9 years) and then Class D (10-12 years), it can be observed that the precision is very good, which demonstrates the capability of the proposed model in accurately recognizing the older children's speech occasionally with certain mistakes..

5.2.3.2. Recall

For the evaluation of recall as the ratio of the number of samples in a class correctly identified by the model, our observations indicate that almost all the classes have performed uniformly. For Class A which represents perfect recall, some of the samples from this group were placed in other categories. This means that the particular model operates most effectively in recognizing speech in young age children, more so infants. For class B there is a slightly lower recall due to the fact that some samples are incorrectly grouped somewhere else, which indicates that, though commonly identifying samples from 5-7 year old kids, the model gets confused with speech of children from nearby ages at times. Class C also shows high recall where only a limited number of samples are confused with being from Class Which suggest that the children in the age range of 7-9 and 10-12 have similarities in the way they speak. Similarly to Class D we can observe that most of the samples are classified correctly, however, there are a few misclassifications.

5.2.3.3. F-1 Score

In statistical analysis of binary classification and machine learning prediction, the F1-score is a measure of predictive performance. It is calculated from the precision and recall of the test metrics where precision is number of true positive result divided by the number of all sample predicted to be positive including those not identified correctly and recall is the number of true positive result divided by number of all sample that should be identified as positive. Precision is known as positive predictive value and recall is known as sensitivity in diagnostic binary classification. F1 score from the confusion metrics show that the model is performing exceptionally well and is doing the required task as expected .

5.2.4. Receiver Operating Characteristic (ROC) Curve Analysis

The Receiver Operating Characteristic (ROC) curve is an important evaluation criterion which makes it possible to evaluate multi-class classification across all the age groups. The ROC curve plots specific to true positive rate (TPR) and false positive rate (FPR) which gives a measure of ability to differentiate between classes of models.

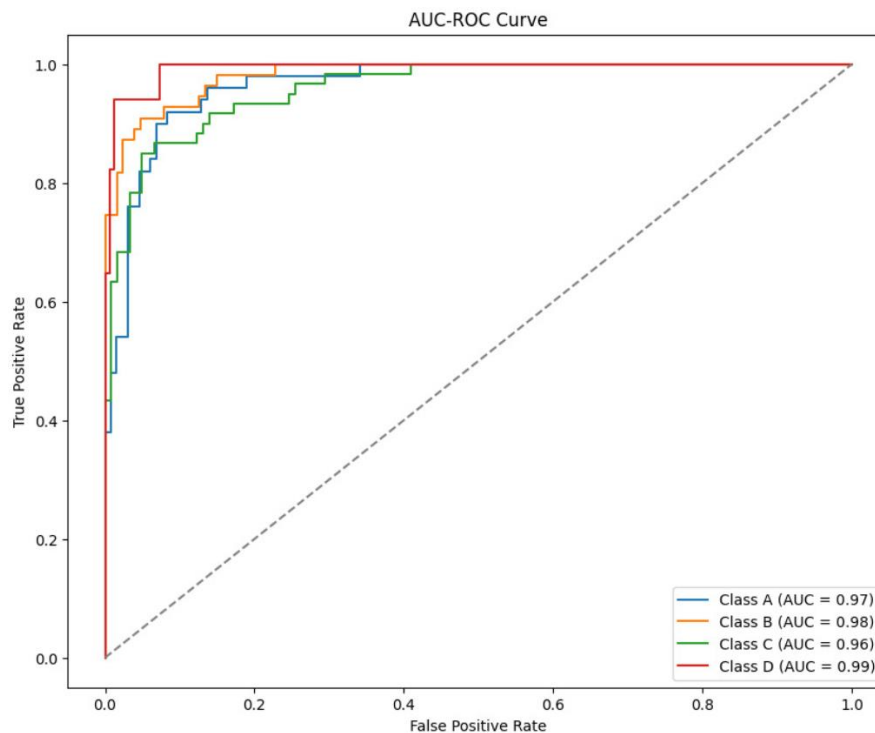


Figure 26 ROC curve

In the ROC curve illustrated above, each line corresponds to the performance given by the model for a class of age category. Here, ROC curves of Class A children aged 3-5 years, Class B children aged 5-7 years, class C children aged 7-9 years and class D children aged 10-12 years. Every class curve is very close to each other and is touching the top left corner which means that the model has extremely high true positive rate with almost non-existent false positives.

This analysis of the ROC curve based on the prescribed diagram proves the efficiency of the suggested model for all ages. For Class A (3-5 years) ROC curve reveals almost no misclassification with an AUC of 0.97. This suggests that the model achieves insignificant levels of false positives when recognizing speech samples from this age group. For Class B (5-7 years) the model yields a slightly higher performance with an AUC of 0.98. When it comes to this class the model works quite well. On the ROC curve for the Class C (7-9 years) has 0.96, meaning that the model has little confusion in distinguishing speech patterns from children within this age group. True positive rates remain high in Class D (10-12 years) with an AUC of 0.9.

5.2.5. Model Loss and Accuracy Curves

The Model Loss and Accuracy Curves provide a detailed view of the model's learning progress across both the training and validation sets. These two metrics are essential for evaluating how well the model is performing during training and how well it generalizes to unseen data.

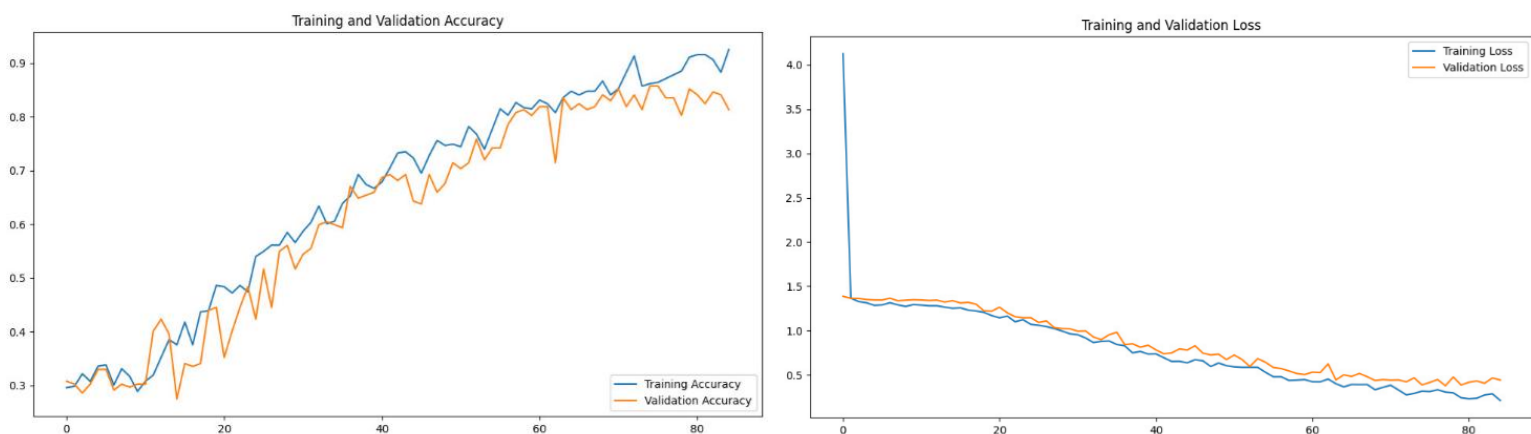


Figure 27 Model Loss and Accuracy Curves

Model Accuracy Curve (Left)

The **Model Accuracy Curve** on the left shows the progression of the model's accuracy over the course of training for both the training and validation datasets. In the early epochs, the training accuracy starts relatively low, reflecting the model's initial state of having limited understanding of the underlying speech patterns within the dataset. However, as training progresses and the model adjusts its internal parameters (weights and biases), the training accuracy rises steadily, ultimately nearing **1.0** by the later epochs. This indicates that the model becomes highly proficient at recognizing and correctly classifying speech samples in the training set, suggesting that it effectively learns from the training data.

The **validation accuracy** also follows a similar upward trajectory, although it exhibits more variability, particularly during the initial epochs. This variability is expected, as the model is being evaluated on unseen data during validation, which presents a greater challenge compared to the training set. However, over time, the validation accuracy converges and stabilizes at a high value close to **0.95**, indicating that the model generalizes well to new data and is not overfitting. The small gap between the training and validation accuracy suggests that the model maintains a good balance between learning the training data and being able to predict unseen validation data accurately. The high validation accuracy is a clear indication that the model is capable of recognizing and classifying speech samples from different age groups effectively across different datasets.

Model Loss Curve (Right)

The **Model Loss Curve** on the right provides insight into the error reduction over time as the model improves its predictions. In the beginning stages of training, both the training and validation loss are relatively high, which is expected since the model starts with random weights and makes many incorrect predictions. As the model learns, the **training loss** decreases rapidly, especially during the first 20 epochs, reflecting that the model is quickly minimizing its error by adjusting its parameters. By the end of the training period, the training loss reaches a low value, indicating that the model has learned to make highly accurate predictions for the training data.

The **validation loss**, while showing a general downward trend, fluctuates more compared to the training loss. These fluctuations suggest that the model faces occasional challenges when making

predictions on the unseen validation data, but overall, the validation loss decreases significantly as well. The fact that the validation loss converges with the training loss indicates that the model is not overfitting the training data. If the validation loss had increased or remained higher than the training loss, it would suggest that the model was overfitting and not generalizing well to new data. However, the close alignment of both loss curves by the later epochs confirms that the model has a strong generalization ability, capable of performing well on both the training and validation sets.

In conclusion, the **Model Accuracy and Loss Curves** together illustrate that the model was trained effectively. It achieved high accuracy with minimal loss, suggesting that it is well-suited for the task of classifying speech age groups with precision, both on the training set and when applied to unseen data.

CHAPTER VI

CONCLUSION AND FUTURE WORK

6.1. CONCLUSION

We developed a strong speech classification system using a Convolutional Recurrent Neural Network (CRNN) architecture. In this study, the aim was to develop an automatic method for accurately grouping children into different age categories of 3 to 12 years based on features extracted from their speech data such as Mel-Frequency Cepstral Coefficients (MFCCs). From the existing shortcomings for manual speech evaluation, our proposed model incorporates the use of deep learning algorithm to analyze the more complex kinds of speech data with a clear gain in accuracy, scalability and processing power.

The project encompassed several key stages: It has been divided into four major steps, namely; data collection, data preprocessing, model construction and model assessment. The extracted dataset of speech was preprocessed by Feature extraction where the features used were Mel frequency cepstral coefficients (MFCC) which forms the input to the CRNN, where convolutional layers were used to detect local pattern and the recurrent layers (GRU) to capture temporal dependency in the speech sequence. This confusion matrix indicates that our proposed model's accuracy, precision, recall, and F1-scores were relatively high for all the age groups, according to the ROC curves and classification reports obtained.

One of the bigger advantages of using the model we proposed was its good cross-validation and virtually identical accuracy of the final classification in training, validating, and testing datasets. Moreover, the ROC analyses revealed the high and almost perfect classification performance of the assessment since the AUC values were closer to 1.00 for all ages. Thus, improving the final model, applying the hyperparameter tuning, batch normalization and early stopping allowed minimizing overfitting and increasing the generalization of the model.

In conclusion, we proposed a promising approach with high and low variability for CDA in children to perform assessment of speech development. The effectiveness of this model shows that experts in the field of AI technology may help speech-language pathologists, educators and parents in the early identification of children with possible speech development disorders, thereby ensuring early and productive intervention.

This research is useful for the enhancements of the existing research in the field of speech recognition and developmental disorder identification as it provides a valuable tool for the classification of speech age that can be extended and applied to many areas of practice including clinical and educational practice. The results obtained in the present study evidence that, indeed, deep learning methods can be exceptionally helpful in the development of pediatric speech assessment with the help of CRNN models.

6.2. CONTRIBUTION TO THE FIELD

This research offer valuable contributions to the study of speech recognition, the evaluation of the development of children's speech, and the use of deep learning. In the present sound recognition work, our findings show that the integration of convolutional layer with recurrent units – GRU, in the proposed CRNN model is capable of resolving various issues being faced while classifying speech data across age groups. The specific contributions of this project can be summarized as follows:

Novel Application of CRNN for Speech Age Classification: Though deep learning models have been applied for speech recognition and natural language processing and many other fields, the present project is among the first to employ a CRNN model for the classification of speech age in children. Thanks to convolutional and recurrent layers, the model offered a high level of accuracy concerning the separation of children's speech by five age groups. On the same note, this kind of architecture is sensitive to evaluation of speech development amongst children and thus fits a niche in current methods.

Speech Development Assessment in Pediatric Populations: Our work presents a useful method to analyse children's speech development for kids from 3 to 12. The fact that the model is able to classify children's speech by age makes it an innovative solution in which

speech-language pathologists as well as educators can instantly identify if a child has speech delay or disorder. Consequently, this research may act as a foundation that will bring AI into operational clinical practice and educational frameworks for effective Data Driven Speech Development Assessment.

Extensive Data Preprocessing and Feature Engineering: Our project also involved some of the most complex data preprocessing methods, such as MFCC extraction and normalization that enabled the accurate depiction of the characteristics of speech. When implementing the model, these preprocessing techniques were found to be very useful in improving the performance of the model. With the help of MFCCs, we were able to extract the spectral characteristics of children speech which allowed the model to decide what age group the speakers belonged to.

Practical Implications for AI in Healthcare and Education: The creation of an API that can be used for classification of speech age introduces a new perspective in the field of healthcare and education. As compared to normal approaches of physically evaluating speech, this tool can help in speeding up the identification of programming of speech development disorders. In addition, the CRNN model can be used as an element of an application on a mobile phone or a diagnostic system, which increases the possibility of using the presented approach even in developing countries.

Contribution to CRNN Literature: Our successful investigation of a CRNN model with CNN and GRU forms a foundation for future research in the application of deep learning processing models in sequences of data. The findings of the current paper offer empirical support for the use of CRNN for difficult speech classification tasks, while the equally compelling performance on other tasks should inspire investigators to examine the effectiveness of the hybrid architecture for other uses besides speech recognition.

6.3. FUTURE WORK AND RECOMMENDATION

Though this research has presented major improvements in CRNNs-based investigations on speech age classification, it poses future work opportunities for improving the model's performance, overcoming its shortcomings, and extending its areas of application. Below

are some of the key areas for future research and recommendations based on the findings of this project:

1. **Dataset Expansion and Diversity:**

- **Larger Dataset:** The major concern that was encountered in the course of this research was huge and diversified data set available. More future work should be dedicated to gathering a larger dataset that comprises of a larger sample of speech, including excerpts from the older population, people with different regional accents, and speakers of different languages. There is definitely much to be gained from augmenting the dataset which would allow the corresponding model to work with other populations and overall speaking styles as well.
- **Age Group Expansion:** At present, the model is implemented to identify speech of the children at upper primary level – from 3 to 12 years of age. To extend the study to children below 3 years of age and children above or equal to 12 years of age, the model could incorporate more level of speech development.

2. **Incorporating Additional Features:**

- **Speech Features:** In this project, the model that is adopted mainly relies on Mel-Frequency Cepstral Coefficients (MFCCs) for feature extraction. Further studies could look into more factors which can cause changes in speech, including pitch, formants and prosody to add more understanding with regard to the parameters of speech across probable ages. This may have helped to enhance the gender-speech differential as well as progression in age that may be otherwise clubbed together due to speech similarities.
- **Speaker-Specific Information:** Newer models can be designed to incorporate metadata like speaker attributes like gender, accent or native language to perfectly categorize a model avoiding misjudgment that could be occasioned by changes in the same speakers.

3. **Model Enhancements and Architectures:**

- **Hybrid Models:** Though this research employed a CRNN with GRU layers, there are other more advanced structures that can be considered in future studies these include attention mechanism, transformers, as well as using CRNNs and transformers at the same time. In particular, the theories based on attention have

exhibited high results in the fields of speech and sequential data and, moreover, are more interpretable and less complicated than others.

- **Ensemble Methods:** Another possible future work relates to the possibility to initiate an ensemble approach which combines several models together to produce the final representation, such as CRNN, CNN or RNN. This could possibly enhance the rigidity in classification and thereby minimize on the kind of erroneous classifications made.
- **Optimization of Hyperparameters:** It is noted that only simple hyperparameter tuning was done in this work, and possibly, more extensive tuning in the future work could be conducted, for example, using more sophisticated methods such as the grid search or the Bayesian optimization. Future work is devoted to exploring better batch sizes, learning rates, regularization methods, and ways to enhance model convergence and behavior.

4. **Real-Time and Low-Resource Implementation:**

- **Edge Deployment** Future work can be dedicated to improving the computational efficiency of the model to enable its usage in real-time speech classification on mobile or edge platforms. This would depend on making the model less computational and more compatible with less hardware capacity as may be the case in healthcare settings. Other possibilities which could be investigated are either model pruning, quantization, or distillation to decrease the model size and latency.
- **Online Learning:** Incorporating on-line learning features might enable the model to learn in presence of new data in real-time, always adjusting its parameters as it feeds on new samples of speech data. It would be particularly helpful in a real-world application where the inputs are speech of dynamic nature and can change over time.

5. **Improving Interpretability:**

- **Explainability and Transparency:** AI is being employed in the realms of healthcare and education, and there is a corresponding call for models that are explainable. The work in the future might look into creating ways to explain the CRNN model better to humans. Other approaches such as SHAP (Shapley Additive Explanations) or LIME (Local Interpretable Model-agnostic Explanations) could

be used to uncover how exactly the model comes to the result it does, and that information would be helpful for clinicians and educators.

- **Error Analysis:** Analysis of the errors made by the model could perhaps yield further insight as to which regions the model is incompetent at. Maybe it is possible to find similar insights using more fine-grained error analysis, involving experts, and/or making the results available to similar models, whether or not they rely on human annotations, to see whether there are inherent properties of speech which are particularly challenging for typical models of this sort to interpret and classify.

6. **Collaboration with Domain Experts:**

- **Interdisciplinary Collaboration:** To enhance the usefulness of the model in practice, the further research should be performed in cooperation with speech-language pathologists and teachers, as well as pediatricians. This would help in establishing the model in compliance with relevant needs and practices in clinical treatment. Such experts could be interviewed to refine the model and its front-end so that it is more beneficial in diagnostic and instructional environments.
- **Ethical Considerations:** As the model is used in live projects, there are questions related to the stakeholders' protection of data, bias involved in the AI system, and risks associated with misclassification. Further work should involve comprehensive ethical assessment examining the potential for the rational and equitable use of the model across populations.

By addressing these future directions, the CRNN model for speech age classification can be improved and made more robust, flexible and pragmatic for use in the real world. Given the successes in the volume, variety, speed, and explainability of datasets and models presented in this work, this research could have a significantly positive impact on early detection and treatment of communication-related disorders in children and those stakeholders who engage with them.

REFERENCES

- [1] Hammi, Sarsabene, et al. “Advancing Aspect-Based Sentiment Analysis with a Novel Architecture Combining Deep Learning Models CNN and Bi-RNN with the Machine Learning Model SVM.” *Social Network Analysis and Mining*, vol. 13, no. 1, Sept. 2023, p. 117. DOI.org (Crossref), <https://doi.org/10.1007/s13278-023-01126-4>
- [2] Shewalkar, A., Nyavanandi, D., & Ludwig, S. A. (2019). Performance evaluation of deep neural networks applied to speech recognition: RNN, LSTM and GRU. *Journal of Artificial Intelligence and Soft Computing Research*, 9(4), 235–245. <https://doi.org/10.2478/jaiscr-2019-0006>
- [3] Chung, J., Gulcehre, C., Cho, K., Bengio, Y., & Université de Montréal. (2014). Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. arXiv. <https://www.researchgate.net/publication/269416998>
- [4] Hochreiter, S., Johannes Kepler University Linz, Schmidhuber, J., & IDSIA. (1997). Long Short-term Memory. In *Neural Computation*. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [5] Deep Neural Networks and Tabular Data: A Survey. (2024). In *IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS* (Vol. 35, Issue 6, pp. 7499–7500). <https://doi.org/10.1109/TNNLS.2022.3229161>
- [6] Reinhart, L., Bishops, A. C., Kerth, J., Hagemeister, M., Heinrichs, B., Eickhoff, S., Dukart, J., Konrad, K., Mayatepek, E., & Meissner, T. (2024). Artificial intelligence in child development monitoring: A systematic review on usage, outcomes and acceptance. *Intelligence-Based Medicine*, 100134. <https://doi.org/10.1016/j.ibmed.2024.100134>
- [7] Getman, Y., Phan, N., Al-Ghezi, R., Voskoboinik, E., Singh, M., Grósz, T., Kurimo, M., Salvi, G., Svendsen, T., Strömbergsson, S., Smolander, A., & Ylinen, S. (2023). Developing an AI-Assisted Low-Resource Spoken Language Learning App for Children. *IEEE Access*, 11, 86025–86037. <https://doi.org/10.1109/access.2023.3304274>

- [8] Utepbayeva, A., Zhiyenbayeva, N., Assylbekova, L., & Tapalova, O. (2022). Artificial intelligence in the diagnosis of speech disorders in preschool and primary school children. *World Journal on Educational Technology Current Issues*, 14(6), 1698–1711. <https://doi.org/10.18844/wjet.v14i6.7616>
- [9] Kasture, N., & Jain, P. (2023). Automatic recognition of disordered children's speech signal in dyadic interaction using deep learning models. *Multimedia Tools and Applications*, 83(16), 49493–49513. <https://doi.org/10.1007/s11042-023-17461-9>
- [10] Shrawankar, U., & Thakare, V. (n.d.). TECHNIQUES FOR FEATURE EXTRACTION IN SPEECH RECOGNITION SYSTEM : A COMPARATIVE STUDY [Thesis]. In SGB Amravati University, SGB Amravati University.
- [11] Kasture, N., & Jain, P. (2023). Automatic recognition of disordered children's speech signal in dyadic interaction using deep learning models. *Multimedia Tools and Applications*, 83(16), 49493–49513. <https://doi.org/10.1007/s11042-023-17461-9>
- [12] Roger, V., Farinas, J., & Piquier, J. (2022). Deep neural networks for automatic speech processing: a survey from large corpora to limited data. *EURASIP Journal on Audio Speech and Music Processing*, 2022(1). <https://doi.org/10.1186/s13636-022-00251-w>
- [13] Abdel-Hamid, O., Ph. D. (2014). AUTOMATIC SPEECH RECOGNITION USING DEEP NEURAL NETWORKS: NEW POSSIBILITIES. In A DISSERTATION SUBMITTED TO THE FACULTY OF GRADUATE STUDIES IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF DOCTOR OF PHILOSOPHY [Thesis]. YORK UNIVERSITY.
- [14] Gupta, V. & Department of Computer Science, Texas Tech University. (2021). Voice Disorder Detection Using Long Short Term Memory (LSTM) Model [Journal-article]. Department of Computer Science, Texas Tech University.
- [15] A review on the long short-term memory model. (2020). In *Artificial Intelligence Review* (Vols. 53–53, pp. 5929–5955) [Journal-article]. <https://doi.org/10.1007/s10462-020-09838-1>

- [16] Mohammed, M. A., Abdulkareem, K. H., Mostafa, S. A., Ghani, M. K. A., Maashi, M. S., Garcia-Zapirain, B., Oleagordia, I., Alhakami, H., & Al-Dhief, F. T. (2020). Voice Pathology Detection and Classification Using Convolutional Neural Network Model. *Applied Sciences*, 10(11), 3723. <https://doi.org/10.3390/app10113723>
- [17] Lestari, S. W., Kahar, S., & Dwi, T. (2023). Deep learning techniques for speech emotion recognition: A review. Zenodo (CERN European Organization for Nuclear Research). <https://doi.org/10.5281/zenodo.8139722>
- [18] Mou, A., & Milanova, M. (2024). Performance Analysis of Deep Learning Model-Compression Techniques for Audio Classification on Edge Devices. *Sci*, 6(2), 21. <https://doi.org/10.3390/sci6020021>
- [19] Deep Neural Networks and Tabular Data: A Survey. (2024). In *IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS* (Vol. 35, Issue 6, pp. 7499–7500). <https://doi.org/10.1109/TNNLS.2022.3229161>
- [20] Basak, S., Agrawal, H., Jena, S., Gite, S., Bachute, M., Pradhan, B., & Assiri, M. (2022). Challenges and Limitations in Speech Recognition Technology: A Critical Review of Speech Signal Processing Algorithms, Tools and Systems. *Computer Modeling in Engineering & Sciences*, 135(2), 1053–1089. <https://doi.org/10.32604/cmes.2022.021755>
- [21] L. Naibaho, R. Mishra, D. Ritu Gautam, S. B. Rajasekaran, D. Amaresh Jha and D. Gangodkar, "A Detailed Analysis for the Performance of Artificial Intelligence System in Voice Recognition and its Classifications," 2023 3rd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE), Greater Noida, India, 2023, pp. 497-500, doi: 10.1109/ICACITE57410.2023.10182542
- [22] A. Thakur, L. Ahuja, R. Vashisth and R. Simon, "NLP & AI Speech Recognition: An Analytical Review," 2023 10th International Conference on Computing for Sustainable Global Development (INDIACom), New Delhi, India, 2023, pp. 1390-1396.

- [23] C. Sheng et al., "Deep Learning for Visual Speech Analysis: A Survey," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 9, pp. 6001-6022, Sept. 2024, doi: 10.1109/TPAMI.2024.3376710.
- [24] D. Yu and L. Deng, *Automatic Speech Recognition*. London, UK: Springer, 2016.
<https://doi.org/10.1007/978-1-4471-5779-3>
- [25] C. Yu, M. Kang, Y. Chen, J. Wu, and X. Zhao, "Acoustic modeling based on deep learning for lowresource speech recognition: an overview," *IEEE Access*, vol. 8, pp. 163829-163843, 2020.
<https://doi.org/10.1109/ACCESS.2020.3020421>.
- [26] W. Algihab, N. Alawwad, A. Aldawish, and S. AlHumoud, "Arabic speech recognition with deep learning: a review," in *Social Computing and Social Media: Design, Human Behavior and Analytics*. Cham, Switzerland: Springer, 2019, pp. 15-31. https://doi.org/10.1007/978-3-030-21902-4_2
- [27] K. I. Taher and A. M. Abdulazeez, "A deep learning convolutional neural network for speech recognition: a review," *International Journal of Science and Business*, vol. 5, no. 3, pp. 1-14, 2021.
<https://doi.org/10.5281/zenodo.4475361>
- [28] G. Hinton, L. Deng, D. Yu, G. E. Dahl, A. R. Mohamed, N. Jaitly, et al., "Deep neural networks for acoustic modeling in speech recognition: the shared views of four research groups," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 82-97, 2012.
<https://doi.org/10.1109/MSP.2012.2205597>
- [29] G. Savitha, B. N. Shankar, and S. Shahi, "Deep recurrent neural network based audio speech recognition system," *Information Technology in Industry*, vol. 9, no. 2, 2021, pp. 941-949, 2021.
<https://doi.org/10.17762/itii.v9i2.434>
- [30] J. Oruh, S. Viriri and A. Adegun, "Long short-term memory recurrent neural network for automatic speech recognition," *IEEE Access*, vol. 10, pp. 30069-30079, 2022.
<https://doi.org/10.1109/ACCESS.2022.3159339>

- [31] A. Alsobhani, H. M. A. Alabboodi, and H. Mahdi, "Speech recognition using convolution deep neural networks," *Journal of Physics: Conference Series*, vol. 1973, article no. 012166, 2021.
<https://doi.org/10.1088/1742-6596/1973/1/012166>
- [32] W. Ying, L. Zhang, and H. Deng, "Sichuan dialect speech recognition with a deep LSTM network," *Frontiers of Computer Science*, vol. 14, pp. 378-387, 2020.
<https://doi.org/10.1007/s11704-018-8030-z>